significant and do not reflect a general prevalence of potentially-sensitive data. $X^2(8, N = 125) = 6.95, p = 0.27$.

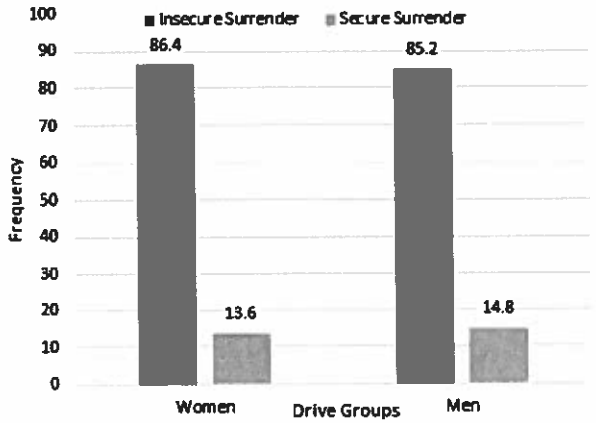

Figure 7. Deletion effectiveness by gender as percentages.

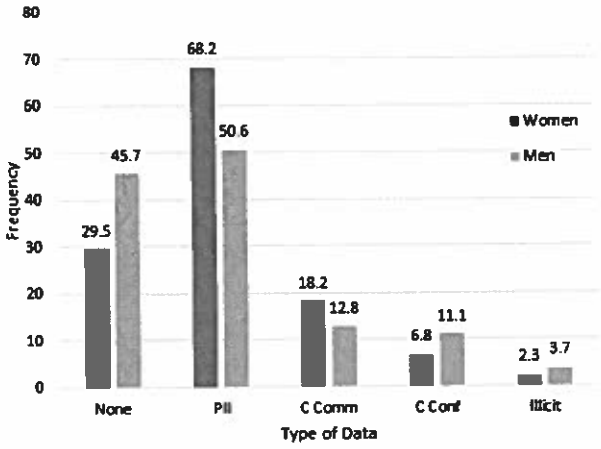

Figure 8. Presence of sensitive data by gender as percentages.

Finally, we see no statistically significant difference in confidence of the participants' deletion methods (see Figure 9). Again, we removed the 36 participants that indicated they did not delete any data.

This leads us to reason that we cannot discern either the method of deletion, the presence of sensitive data, or the confidence the participant will have in their deletion methods from their gender.

5.3.2 Age. To create more even groups in terms of group population while maintaining similar age-bands, we combined the groups from the survey into the following categories: 18–30, 31–40, 41–50, and Over 51. Even with this adjustment, 75/126 = 59.52% of the participants identified themselves as 18–30 (see Table 5). As in the previous analysis, we found no statistically significant differences in age with regard to deletion method, ($X^2(9, N = 126) = 4.03, p = 0.09$, see Figure 10), presence of sensitive data ($X^2(3, N = 126) = 1.31, p = 0.27$, see Figure 11) nor confidence in deletion methods $X^2(12, N = 90) = 3.99, p = 0.09$, see Figure 12). In Figure 12, we once again excluded participants that indicated they did not remove data.

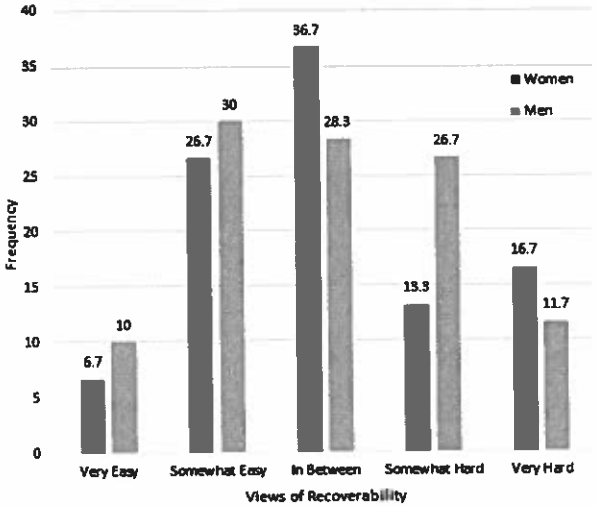

Figure 9. Confidence of deletion by gender as percentages.

Table 5. Total of Participants by Age Bracket

Age bracket	Total
18 – 30	75
31 – 40	19
41 – 50	14
Over 50	18

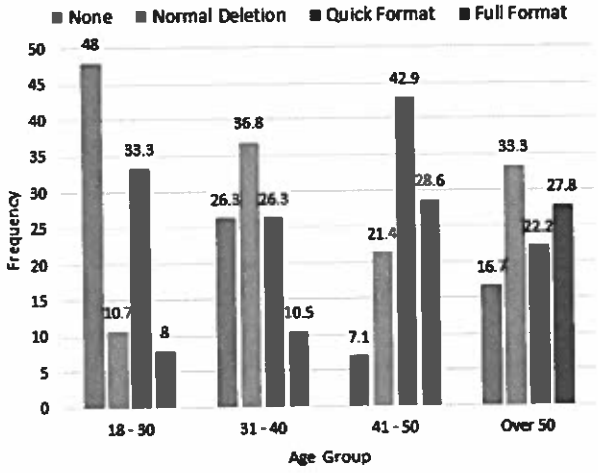

Figure 10. Deletion method with regard to age as percentages.

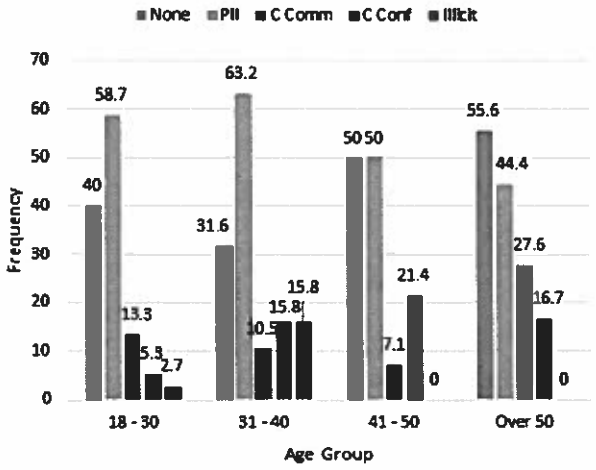

Figure 11. Presence of sensitive data by age as percentages.

5.3.3 Profession/Field of Study. As with gender, we compared the deletion methods, presence of sensitive data, and confidence in deletion method to the pre-determined profession/field-of-study categories. The reader should note that we only asked participants to select a profession/field-of-study, not status as a student or university employee.

Because an overwhelming majority of participants self-identified as being in the one category, namely Science/Engineering category (n = 60, see Table 6), statistical analysis was problematic. Thus, we did not include analysis in this paper.

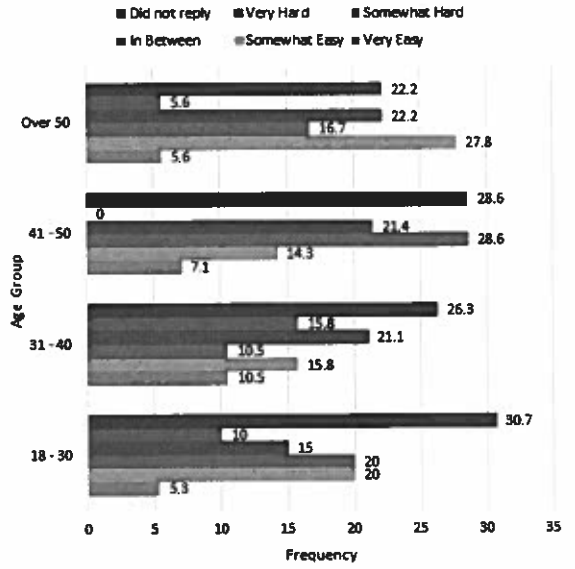

Figure 12. Confidence of deletion by age as percentages.

Table 6. Total of Participants by Profession/Field of Study

Profession/FoS	Total
Humanities	37
Arts	9
Science/Engineering	60
Business	16
Other/Did not respond	4

5.4 Limitations and Delimitations

Due to scarcity of resources, we encountered manpower limitations, impacting our ability to parse drives. This in turn affected the number of drives processed. The singular buyback study location, which was physically close to traditionally-STEM departments at a university, skewed the participant field of study. The buyback study could have been held in different and/or multiple locations (such as a mall, public recreation center, business environment, etc.), but we chose to hold it in a university due to resource constraints. Also, conducting the survey in a Midwestern university narrowed our pool of potential buyback participants, as many of the participants were associated with the university.

A delimitation of our study included buying drives from a combination of individual drives and lots of drives. We chose to purchase some lots of drives to help minimize the cost of the study. Those lots demonstrated extreme diversity in drive model, size, and other advertised attributes. In fact, we only bought lots in which all thumb drives were different in some way. We also limited our purchase of market drives to eBay and Amazon Marketplace; however, other avenues to purchase used drives do exist. One example is to purchase data that happens to be distributed on a thumb drive, such as conference proceedings! We also chose to use the TestDisk [44] suite of forensics software; however other researchers may have chosen to use different open-source or commercially-available forensics software. We chose this software due to its ease-of-use with the research team. We chose to process only English-language drives due to the language skills of our hired researchers, and we chose to process only FAT-formatted drives to streamline our low-level file system forensics procedures.

6 DISCUSSION AND FUTURE WORK

The lack of relationship between users' perception of data recovery and method of deletion may arise from any number of factors. We list some possibilities here:

- Lack of awareness in the need to use strong deletion methods.
- Lack of technical understanding concerning the use of strong deletion methods.
- Perceived difficulty in using strong deletion methods.

One potential mitigation strategy is to better educate the public on the use of strong deletion methods, such as full formatting or the use of specialized tools such as DBAN. However, this strategy hinges on the users' willingness to learn about additional techniques [19]. Another strategy is to set strong deletion methods to be the default. One current example of "security by default" is

the trend of mobile phone operating systems to encrypt the phone storage by default [2, 3]. If strong deletion were made the default choice, perhaps users would choose strong deletion more often. One example might be setting the full format as the default format setting and making this option easier to find. Per-file strong (or secure) deletion is more difficult [41, 40, 10, 46, 34, 35, 9, 36], and this option would likely require modifications to the applicable operating system and storage interface.

Even though many organizations include policies for sanitization of removable storage media like thumb drives, thumb drives are often difficult to track. A work thumb drive could easily wind up in an employee's home or in a lost and found, where it may not be properly sanitized upon surrender. As our special cases illustrate, organizations are not immune to this problem. One policy solution is to ban the use of thumb drives all together—however, users may not be willing to give up the convenience a thumb drive provides, and thus the policy may be difficult to enforce.

4. Physical destruction could properly sanitize the thumb drive, but only if the actual NAND chip itself is destroyed. Phillips et al. [32] discuss interesting ways in which thumb drives can be physically destroyed and whether or not data could be recovered in those instances. The downside to physical destruction is that the drive may never be used again. Another solution could be to modify the thumb drive to allow some sort of secure deletion button, slider bar, or reset pin. Another technique could be to modify the thumb drive to allow a secure erase command, much like hard drives and solid state drives [20]. These solutions would allow the thumb drive to remain usable after sanitization, but the solutions must be implemented correctly to be of use [46].

Finally, users could 1) use software to encrypt data on their thumb drives or else 2) use self-encrypting thumb drives [21]. The first suggestion is less user-friendly, as the software needed to decrypt the drive must be installed on every computer. The second suggestion is often more expensive. Also, if the key is ever recovered after device surrender (e.g. through social engineering or a brute-force attack on the password used to generate the key), the data on the drive would no longer be deleted.

During sensitive data classification, we found classifying movies, TV shows, and music as illicit (illegal) is difficult for a number of reasons. First, laws and regulations regarding copyrighted materials vary [13, 25]. Secondly, it becomes troublesome to determine if a particular music, TV show, or movie file were actually purchased or archived legitimately. Sometimes, one can infer by the naming or packaging of a file if it likely came from an illegal source, but it is difficult to infer illegality beyond reasonable doubt. Thus, we did not categorize copyrighted works as illicit in this research. Using advanced methods to determine legality of media files will be future work.

Although the wording on our consent form does not employ any deception, we considered using stronger language to imply the type of forensics we would employ on the buyback drives. However, a main focus of this study was to attempt to simulate the behavior of the broader population of people that sell drives and devices in marketplaces on the Internet, and those marketplaces do not include such a warning. Including a stronger warning might have biased the study. We worked diligently with our IRB department to provide strong protections to participants in our buyback study and any data recovered, and those procedures are detailed in Section 4.2. In addition, participants having any further concerns were provided with information necessary to contact the administrator of the study.

We have many avenues for future work. In light of the Snowden NSA revelations [26] and the recent high-profile clash between Apple and the U.S. government concerning the unlocking of an iPhone [4], it may be interesting to re-run the buyback program to see if any of the media attention has affected deletion habits and perceptions. The survey could also be expanded to ask more questions, including topics such as:

- Use of encryption to protect potentially-sensitive information.
- What types of information participants deem sensitive.

Other future work includes expanding the buyback study to different pools of participants in different locations, as well as using hardware forensics methods to remove the NAND chips from the USB drive to read all storage areas on the chip [5].

7 CONCLUSION

To understand the deletion beliefs and practices of people surrendering thumb drives in exchange for money or goods, we conducted a buyback program with a survey component and compared the results to drives obtained through common markets such as eBay and Amazon Marketplace. Of interest, we found:

- The percentage of buyback and market drives that contain potentially-sensitive data in our study to be over 60%.
- When comparing the market drives to the buyback drives in terms of method of deletion, presence of potentially-sensitive data, and spread of sensitive data by drive, we found that the groups are not dissimilar. This may indicate further similarities between our buyback study and the wider world.
- No statistically significant relationship connecting the presence of potentially-sensitive data and deletion method.
- No correlation between users' perceived versus actual effectiveness of deletion in the buyback group
- Near equal ratios of men and women surrendered drives with recoverable data in the buyback group.
- Used drives in our study contained a variety of sensitive data, including employee databases, tax returns, internal corporate documents, scanned passports, and illicit photos.

We found a general state of confusion about the effectiveness of deletion methods. Future work must be performed to remedy the situation, perhaps by better privacy awareness in conjunction with more default secure sanitization policies in operating systems, built-in device mechanisms, or some combination of these solutions.

REFERENCES

[1] 1 "terrific employee" + 1 thumb drive + 6,000 lost medical records = fired! https://nakedsecurity.sophos.com/2013/01/21/1-terrific-employee-1-thumb-drive-6000-lost-medical-records-fired/. Accessed: 2015-03-04.

[2] Android L will offer default encryption just like iOS 8: http://www.zdnet.com/article/android-l-will-offer-default-encryption-just-like-ios-8/. Accessed: 2015-03-08.

[3] Apple expands data encryption under iOS 8, making handover to cops moot: 2014. http://arstechnica.com/apple/2014/09/apple-expands-data-encryption-under-ios-8-making-handover-to-cops-moot/. Accessed: 2015-03-08.

[4] Apple vows to resist FBI demand to crack iPhone linked to San Bernardino attacks - The Washington Post: 2016. https://www.washingtonpost.com/world/national-security/us-wants-apple-to-help-unlock-iphone-used-by-san-bernardino-shooter/2016/02/16/69b903ee-d4d9-11e5-9823-02b905009f99_story.html. Accessed: 2016-03-03.

[5] Breeuwsma, M. et al. 2007. Forensic data recovery from flash memory. Small Scale Digital Device Forensics Journal. 1, 1 (2007), 1–17.

[6] Cisco Survey Designates 10 Riskiest Data Loss Behaviors: 2008. http://www.crn.com/news/security/210604893/cisco-survey-designates-10-riskiest-data-loss-behaviors.htm. Accessed: 2012-08-07.

[7] CITI - Collaborative Institutional Training Initiative: https://www.citiprogram.org/index.cfm?pageID=88. Accessed: 2015-06-01.

[8] Cooke, J. 2007. Flash memory technology direction. Micron Applications Engineering Document. (2007).

[9] Diesburg, S. et al. 2012. TrueErase: Per-file Secure Deletion for the Storage Data Path. Proceedings of the 28th Annual Computer Security Applications Conference (Dec. 2012).

[10] Diesburg, S.M. and Wang, A.-I.A. 2010. A survey of confidential data storage and deletion methods. ACM Computing Surveys. 43, 1 (Dec. 2010), 2:1–2:37.

[11] Fair and Accurate Credit Transactions Act of 2003: 2003. http://www.gpo.gov/fdsys/pkg/PLAW-108publ159/html/PLAW-108publ159.htm. Accessed: 2015-03-12.

[12] Garfinkel, S.L. and Shelat, A. 2003. Remembrance of data passed: a study of disk sanitization practices. Security Privacy, IEEE. 1, 1 (Feb. 2003), 17 – 27.

[13] Goldstein, P. 2001. International Copyright: Principles, Law, and Practice. Oxford University Press.

[14] Gramm-Leach-Bliley Act: 1999. http://www.gpo.gov/fdsys/pkg/PLAW-106publ102/html/PLAW-106publ102.htm. Accessed: 2015-03-12.

[15] Gutmann, P. 2001. Data remanence in semiconductor devices. 10th USENIX Security Symposium (2001).

[16] Gutmann, P. 1996. Secure deletion of data from magnetic and solid-state memory. Proceedings of the Sixth USENIX Security Symposium, San Jose, CA (1996).

[17] Health Insurance Portability and Accountability Act of 1996: 1996. http://www.hhs.gov/ocr/privacy/hipaa/administrative/statute/hipaastatutepdf.pdf. Accessed: 2012-07-24.

[18] hexedit(1) - Linux man page: http://linux.die.net/man/1/hexedit. Accessed: 2015-03-05.

[19] Howe, A.E. et al. 2012. The Psychology of Security for the Home Computer User. 2012 IEEE Symposium on Security and Privacy (SP) (May 2012), 209–223.

[20] Hughes, G. 2004. CMRR Protocols for disk drive secure erase. Technical report, Center for Magnetic Recording Research, University of California, San Diego.

[21] Ironkey: 2015. http://www.ironkey.com. Accessed: 2015-06-01.

[22] Jones, A. et al. 2010. The 2009 analysis of information remaining on disks offered for sale on the second hand market. (2010).

[23] Jones, A. et al. 2009. The 2009 Analysis of Information Remaining on USB Storage Devices Offered for Sale on the Second Hand Market. Australian Digital Forensics Conference (2009), 61.

[24] King, C. and Vidas, T. 2011. Empirical analysis of solid state disk data retention when used with contemporary operating systems. Digital Investigation. 8, (2011), S111–S117.

[25] Küng, L. et al. 2008. The Internet and the Mass Media. SAGE.

[26] Landau, S. 2013. Making Sense from Snowden: What's Significant in the NSA Surveillance Revelations. IEEE Security & Privacy.

[27] McCallister, E. et al. 2010. Guide to Protecting the Confidentiality of Personally Identifiable Information (PII) Special Publication 800-122. National Institute of Standards and Technology.

[28] Microsoft Corporation 2000. Microsoft Extensible Firmware Initiative FAT32 File System Specification.

[29] National Industrial Security Program Operating Manual 5220.22-M: 1995. http://www.usaid.gov/policy/ads/500/d522022m.pdf. Accessed: 2012-07-26.

[30] NC community college employee loses thumb drive with personal info of thousands of former students: http://myfox8.com/2015/02/27/nc-community-college-employee-loses-thumb-drive-with-personal-info-of-thousands-of-former-students/. Accessed: 2015-03-03.

[31] One in 10 at risk of hard drive identity theft: 2012. http://www.telegraph.co.uk/technology/news/9224163/One-in-10-at-risk-of-hard-drive-identity-theft.html. Accessed: 2015-03-04.

[32] Phillips, B.J. et al. 2008. Recovering Data from USB Flash Memory Sticks That Have Been Damaged or Electronically Erased. Proceedings of the 1st International Conference on Forensic Applications and Techniques in Telecommunications, Information, and Multimedia and Workshop (ICST, Brussels, Belgium, Belgium, 2008), 19:1–19:6.

[33] PhotoRec - Digital Picture and File Recovery: http://www.cgsecurity.org/wiki/PhotoRec. Accessed: 2015-03-06.

[34] Reardon, J. et al. 2012. Data Node Encrypted File System: Efficient Secure Deletion for Flash Memory. 21st USENIX Security Symposium (Aug. 2012).

[35] Reardon, J. et al. 2013. Secure Data Deletion from Persistent Media. Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security (New York, NY, USA, 2013), 271–284.

[36] Reardon, J. et al. 2013. SoK: Secure Data Deletion. 2013 IEEE Symposium on Security and Privacy (May 2013).

[37] Sansurooah, K. and Szewczyk, P. 2012. A Study of Remnant Data Found on USB Storage Devuces Offered for Sale on the Australian Second Hand Market in 2011. 10th Australian Information Security Management Conference (2012), 82.

[38] Sarbanes-Oxley Act of 2002: 2002. http://www.gpo.gov/fdsys/pkg/PLAW-107publ204/html/PLAW-107publ204.htm. Accessed: 2015-03-12.

[39] Shu, F. and Obr, N. 2007. Data set management commands proposal for ATA8-ACS2.

[40] Sivathanu, G. et al. 2006. Type-safe disks. Proceedings of the 7th Symposium on Operating Systems Design and Implementation (Berkeley, CA, USA, 2006), 15–28.

[41] Sivathanu, M. et al. 2003. Semantically-smart disk systems. Proceedings of the 2nd USENIX Conference on File and Storage Technologies (2003), 73–88.

[42] Soto, J. and Bassham, L. 2000. Randomness testing of the advanced encryption standard finalist candidates. National Institute of Standards and Technology.

[43] Special Publication 800-88: Guidelines for Media Sanitization: 2006. http://csrc.nist.gov/publications/nistpubs/800-88/NISTSP800-88_with-errata.pdf. Accessed: 2012-07-26.

[44] TestDisk - Partition Recovery and File Undelete: http://www.cgsecurity.org/wiki/TestDisk. Accessed: 2015-03-06.

[45] Valli, C. and Woodward, A. 2007. Oops they did it again: The 2007 Australian study of remnant data contained on 2nd hand hard disks. Australian Digital Forensics Conference (2007), 12.

[46] Wei, M. et al. 2011. Reliably erasing data from flash-based solid state drives. Proceedings of the 9th USENIX Conference on File and Storage Technologies (Berkeley, CA, USA, 2011), 8–8.

APPENDIX

Below is the survey we gave to our buyback participants. We did not use answers from questions 4, 6, and 7.

1. Did you delete data from your USB thumb drive? ☐ Yes ☐ No

2. (Only answer if you checked no on question 1.) If you didn't delete data from your USB thumb drive, why not?

_____

3. (Only answer these questions if you checked yes on question 1.)

3a. If you deleted your data, please describe the method you used.

_____

3b. How easy was it to delete your data?

☐ Very Easy ☐ Somewhat Easy ☐ In Between ☐ Somewhat Hard ☐ Very Hard

3c. Do you think the deleted data is easy or difficult to recover?

☐ Easy to Recover ☐ Somewhat Easy ☐ In Between ☐ Somewhat Hard ☐ Hard to Recover

4. Estimate how many USB thumb drives you own and use.

Own? _____ Use? _____

5. How much do you care about permanently deleting data on USB thumb drives?

☐ Never Care ☐ Rarely Care ☐ In Between ☐ Often Care ☐ Always Care

6. What is the maximum amount of time you would spend trying to delete your data permanently on a USB thumb drive? _____

7. Which category best describes your field of study or work from the following:

☐ Humanities ☐ Arts ☐ Science/Engineering ☐ Business/Management ☐ Other (Please write):

8. What is your gender?

☐ M ☐ F ☐ Trans* ☐ Fill in _____ ☐ Prefer not to answer

9. What is your age group?

☐ 18-24 ☐ 25-30 ☐ 31-35 ☐ 36-40 ☐ 41-45 ☐ 46-50 ☐ 51-55 ☐ 56-60 ☐ 61-65 ☐ 66+ ☐ Prefer not to answer

Ghosts of Deletions Past: New Secure Deletion Challenges and Solutions

SARAH DIESBURG, University of Northern Iowa

Is secure deletion of data still a problem?

1. INTRODUCTION

The issue of data privacy is becoming increasingly important. While most articles and news pieces concentrate on private information leaked or gathered from the Internet, users must not forget about the difficult issue of preserving privacy on their own electronic devices. In particular, one important aspect of privacy is having control over deletion of data.

The act of completely and irrevocably getting rid of data is called ***secure deletion***. Secure deletion is a difficult action to get right, and unfortunately the advancement of storage devices and operating systems has not made this any easier. With the exception of physical destruction (e.g. smelting, degaussing, acid bath, etc.), software methods may be prone to failure. For example, on-disk secure erase or factory reset commands may report success but actually fail [10] if implemented incorrectly. Some storage devices may not come with any on-disk erase commands at all—think of the thumb drive in the desk drawer! Common device and partition-wide secure deletion programs (such as DBAN [2]) may not erase bad-sector areas on hard drives (HDDs) or non-accessible physical locations on NAND flash devices. Examples of NAND flash devices include thumb drives, solid state drives (SDDs), and smart phone storage.

Full-disk encryption is often touted as a way to confidentially protect data and indirectly solve the secure deletion problem if the device is lost or confiscated. However, this situation is not the same as secure deletion, because the data has not been irrevocably erased. Data can be reinstated if the device is returned to the user. Worse, users of encrypted data may be subjected to coercion for the encryption key, or the device may be subject to decryption or password cracking attempts to restore the password-generated key.

If users would just like to securely delete a few files while keeping the device usable, the field narrows further. Software erasure programs may miss erasure of important metadata, such as file names, file ownership, or access times. File system solutions largely rely on the existence of a magnetic-platter hard disk drive and may not erase files from NAND flash devices or parity on RAID systems [6, 7]. Full-disk encryption also cannot work, as the deletion of the single disk key would render all files useless. Using multi-key encryption may work, but only with the added complications of a key store. Of course, encryption keys must also be securely deleted, which again may not be guaranteed due to the problems listed above. This article addresses some of the current challenges surrounding secure deletion and directions for the future.

2. SECURE DELETION CHALLENGES

Operating systems exhibit a natural tension with secure deletion. Three main themes emerge in mainstream operating systems:

1. Operating systems are optimized for speed.
2. Operating systems are optimized for reliability.
3. Operating systems are built in layers on the principle of information-hiding (see Fig. 1), so vital information needed for secure deletion is not shared.

Software at the application layer often makes swap and temporary files (reliability mechanism). Secure deletion solutions at this layer can fail because they cannot gain access to file system metadata for deletion (information-hiding mechanism), and they

5 cont
many not delete copies of data made at any lower layer (such as a file system journal, or extra copies of data made on devices which use NAND flash).

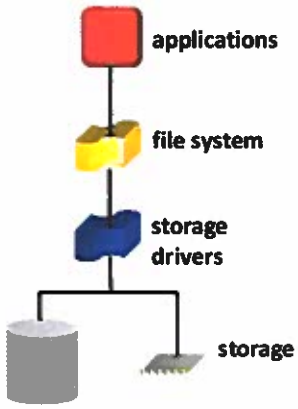

Fig. 1. Operating system storage data path layers.

6. The file system layer often journals copies of file and metadata information (reliability mechanism). Regular deletion mechanisms at the file system level only cause writes or updates to certain metadata, such as the addition of a deletion flag and an update to a free block bitmap (speed mechanism). General-purpose file systems that have built-in secure deletion commands tend to assume traditional hard drive storage and rely on disk overwrite commands, which may fail for NAND flash storage.

At the driver layer, RAID and other virtual devices often duplicate data (both reliability and speed mechanisms). Further complicating potential secure deletion solutions, the applications and file system layers cannot pass vital information to the driver layer, such as information needed to discern between metadata and file data, or even the notion that a deletion is occurring (information-hiding mechanism). After all, a regular operating system deletion just translates into a series of updates (writes) to metadata, so the lower driver layers only process a series of write commands.

7.

Storage devices themselves also complicate secure deletion. HDDs and SSDs contain bad location lists, and these locations cannot be erased. The user may not be able to see what data are in these lists without resorting to hardware forensics. However, devices which contain NAND flash are even worse. Some NAND flash background may be in order to fully understand the problem.

3. NAND FLASH

NAND flash is a type of memory storage with an interface that accepts read, write, and erase requests. NAND has the following characteristics: (1) In-place updates are generally not allowed[1]—once a location is written, the location must be erased before it can be written again; (2) NAND reads and writes are in ***flash pages*** (e.g., 2–8 KiB), but erasures are performed in ***flash blocks*** (e.g., 64–512 KiB consisting of contiguous pages); (3) writing is slower than reading, and erasure can be more than an order of magnitude slower; (4) each storage location can be erased only 10K–1M times; and (5) many NAND flash drives have extra non-addressable physical locations that may hold additional information. One study found NAND flash to hold as much as 6%–25% of additional unreported storage area [10].

The challenges posed by these characteristics are commonly solved through the addition of a ***flash translation layer (FTL)***. The FTL is usually located in hardware and exports a typical hard drive interface instead of the raw flash interface described

[1] Flash overwrites might be allowed for some special cases such as marking a page invalid.

above. As a common optimization to mask slow writes and erases, when flash receives a request to overwrite a flash page, the FTL remaps the write to a pre-erased flash page using a translation table, stamps the page (with a version number), and marks the old page as invalid, to be erased later. Information on invalid pages could be read by removing the NAND flash chip and placing it in a universal reader [1]. To prolong the lifespan of the flash, ***wear-leveling*** techniques are often used to spread the number of erasures evenly across all storage locations. Fig.2 below demonstrates the problem by illustrating typical FTL behavior when a user wishes to overwrite a sensitive area one or more times with patterns and/or random data [4].

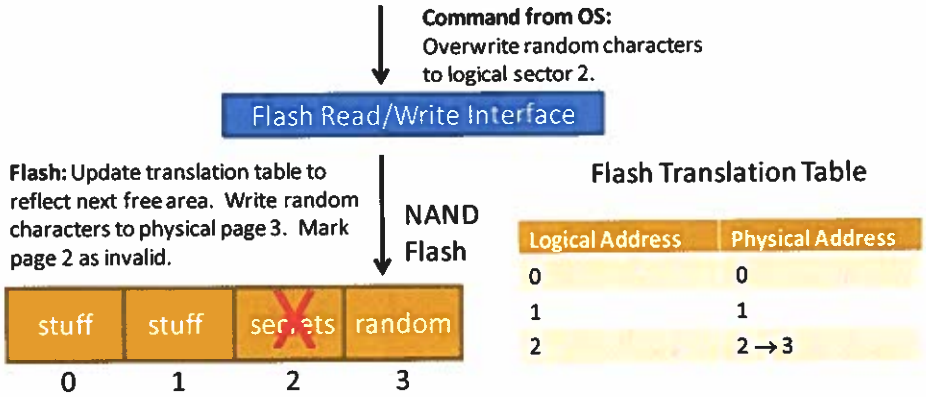

Fig 2. Illustration of an attempted secure deletion using overwrites.

If a partition- or disk-wide overwriting program like DBAN is used, the problem still persists due to the extra storage areas built into the NAND flash. Overwritten pages are marked invalid and may be moved into the inaccessible, extra page range to speed wear-leveling before erasure. Even parts of files overwritten during normal file modification may wind up in the extra page range. Due to the proprietary nature of the FTL algorithms, it is not known exactly when those pages will be erased.

4. FUTURE DIRECTIONS

The secure deletion problem cannot feasibly be solved through increased user awareness. For example, users (even non-technical ones) must be aware of the physical characteristics of their storage devices, complications imposed by their file systems, and limitations imposed by software solutions to choose the best option from the myriad array of commercial, free, and built-in secure deletion products available.

Perhaps the secure deletion problem is instead an engineering problem. The current user model is to make users expend effort and use often advanced tools to securely delete files. *If the default deletion behavior is secure deletion, then users would have to use extra effort to turn secure deletion off.*

Storage encryption may be able to solve the secure deletion problem in the circumstance that the encryption key is completely destroyed, the encryption method is strong, and the implementation of the encryption does not contain errors or back doors. If keys are saved on the storage, then they also must be securely erased.

Operating systems could be redesigned to make secure deletion easier. One step in the right direction is to enable the file system to pass an erasure command all the way down to the storage driver [3, 8]. The NAND flash TRIM command [9] illustrates this full-data-path behavior by allowing the file system to specify blocks no longer in use so that the FTL may garbage collect them at a later time. In its current form, TRIM is implemented for SSD performance.

If a command like TRIM could be adapted for secure deletion, its implementation and use would need some modifications. (1) Deletion must happen immediately, or else, at a guaranteed, verifiable time in the near future. Also, all pages that hold copies of this data must be tracked down and securely deleted. An example of a stack-based FTL that automatically tracks old versions of pages/blocks is INFTL, and such an FTL design could be leveraged for secure deletion [3]. To decrease device wear, an encryption key could be securely deleted in lieu of data [8], or only certain files could be securely deleted (such as in a user's home directory). (2) Metadata must also be deleted, including metadata in a file system journal. Currently, TRIM-enabled file systems tend to concentrate on data blocks. (3) Other storage devices (like HDDs) should also recognize the command and implement suitable deletion.

The new eMMC storage sanitize command [5] is a step in the right direction and will remove data from the unmapped eMMC data regions. However, unless verification of secure deletion is possible, this solution and the above proposed solutions are moot. *Security by obscurity should not be the norm in our increasingly-complex storage devices.* One solution may be to create open and verifiable storage so that all contents on storage could be viewed and encryption algorithms (if any) could be verified. The Linux Memory Technology Device (MTD) subsystem contains software FTL support and access to the raw storage areas through a character device. Drives with hardware FTLs, such as SSDs, could be extended to export a similar, read-only interface to view the raw contents of data areas on the SSD without exposing proprietary microcode or FTL management areas.

In summary, secure deletion is still a hard problem primarily due to operating system and storage design. However, it is a solvable problem for designers of operating systems and storage devices if (1) deletion behavior defaults to secure deletion, (2) operating systems are designed to share information about deletions to storage drives, and (3) storage devices are designed in an open and verifiable manner.

REFERENCES

[1] Breeuwsma, M. et al. 2007. Forensic data recovery from flash memory. *Small Scale Digital Device Forensics Journal*. 1, 1 (2007), 1–17.

[2] Darik's Boot And Nuke | Hard Drive Disk Wipe and Data Clearing: 2012. *http://www.dban.org/*. Accessed: 2012-09-25.

[3] Diesburg, S. et al. 2012. TrueErase: Per-file Secure Deletion for the Storage Data Path. *Proceedings of the 28th Annual Computer Security Applications Conference* (Dec. 2012).

[4] DoD, D.O.E. and CIA, N. 1995. *DoD 5220.22-m, National Industrial Security Program Operating Manual*. January.

[5] JEDEC Solid State Technology Association 2015. Embedded Multi-Media Card (e•MMC) Electrical Standard (5.1) JESD84-B51.

[6] Joukov, N. and Zadok, E. 2005. Adding secure deletion to your favorite file system. *Security in Storage Workshop, 2005. SISW '05. Third IEEE International* (Dec. 2005), 8 pp.–70.

[7] Peterson, Z.N.J. et al. 2005. Secure deletion for a versioning file system. *Proceedings of the USENIX Conference on File And Storage Technologies (FAST)* (2005), 143–154.

[8] Reardon, J. et al. 2012. Data Node Encrypted File System: Efficient Secure Deletion for Flash Memory. *21st USENIX Security Symposium* (Aug. 2012).

[9] Shu, F. and Obr, N. 2007. *Data set management commands proposal for ATA8-ACS2*.

[10] Wei, M. et al. 2011. Reliably erasing data from flash-based solid state drives. *Proceedings of the 9th USENIX Conference on File and Stroage Technologies* (Berkeley, CA, USA, 2011), 8–8.

TrueErase: Leveraging an Auxiliary Data Path for Per-File Secure Deletion

SARAH DIESBURG, University of Northern Iowa
CHRISTOPHER MEYERS, MARK STANOVICH, and AN-I ANDY WANG,
Florida State University
GEOFF KUENNING, Harvey Mudd College

One important aspect of privacy is the ability to securely delete sensitive data from electronic storage in such a way that it cannot be recovered; we call this action *secure deletion*. Short of physically destroying the entire storage medium, existing software secure-deletion solutions tend to be piecemeal at best – they may only work for one type of storage or file system, may force the user to delete all files instead of selected ones, may require the added complexities of encryption and key storage, may require extensive changes and additions to the computer's operating system or storage firmware, and may not handle system crashes gracefully.

We present TrueErase, a holistic secure-deletion framework for individual systems that contain sensitive data. Through design, implementation, verification, and evaluation on both a hard drive and NAND flash, TrueErase shows that it is possible to construct a per-file, secure-deletion framework that can accommodate different storage media and legacy file systems, require limited changes to legacy systems, and handle common crash scenarios. TrueErase can serve as a building block by cryptographic systems that securely delete information by erasing encryption keys. The overhead is dependent on spatial locality, number of sensitive files, and workload (computational- or I/O-bound).

Categories and Subject Descriptors: D.4.2 [**Storage Management**]: Allocation/Deallocation Strategies; D.4.3 [**File Systems Management**]: Access Methods; D.4.6 [**Security and Protection**]: Information Flow Controls

General Terms: Design, Security

Additional Key Words and Phrases: Secure deletion, assured deletion, file systems, storage, security, NAND flash

ACM Reference Format:
Sarah Diesburg, Christopher Meyers, Mark Stanovich, An-I Andy Wang, and Geoff Kuenning. 2016. TrueErase: Leveraging an auxiliary data path for per-file secure deletion. ACM Trans. Storage 12, 4, Article 18 (May 2016), 37 pages.
DOI: http://dx.doi.org/10.1145/2854882

This work is supported by NSF grants CNS-0845672/CNS-1065127, DoE grant P200A060279, Philanthropic Educational Organization, FSU Research Foundation, and UNI Research Foundation. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF, DoE, PEO, FSU, or UNI.
Authors' addresses: S. Diesburg, 311 Innovative Teaching and Technology Center, University of Northern Iowa, Cedar Falls, IA 50614-0507; email: sarah.diesburg@uni.edu; C. Meyers and M. Stanovich, 253 Love Building, Florida State University, Tallahassee, FL 32306-4530; emails: {meyers, stanovic}@cs.fsu.edu; A. I. A. Wang, 269 Love Building, Florida State University, Tallahassee, FL 32306-4530; email: awang@cs.uni.edu; G. Kuenning, Department of Computer Science, Harvey Mudd College, 301 Platt Blvd., Claremont, CA 91711; email: geoff@cs.hmc.edu.

DOI: http://dx.doi.org/10.1145/2854882

1. INTRODUCTION

As the cost of electronic storage declines, more data are stored on electronic media. In 2013, the U.S. Census Bureau reported that 83.8% of U.S. households own a computer (e.g., desktop computers, laptops, smart phones, and other handheld devices computers [United States Census Bureau 2014]). According to surveys conducted by the Center for Research on Information Technology and Organizations at the University of California, Irvine, home users in particular are increasingly using their computers to perform sensitive tasks [Venkatesh et al. 2011]. Throughout the 11 years of this study, usage of email, online shopping, and online banking have increased 26%, 63%, and 150%, respectively. These statistics suggest that many computers cache or store data on personal finances and online transactions, not to mention other confidential data such as tax records, passwords for bank accounts and email.

More sensitive data is being stored within industries and governments as well. Examples of regulation driving the increased storage and deletion of sensitive data include The Health Information Technology for Economic and Clinical Health (HITECH) act [U.S. Department of Health & Human Services 2009], The Health Insurance Portability and Accountability Act of 1996 (HIPPA) [U.S. Department of Health & Human Services 1996], The Fair and Accurate Credit Transactions Act of 2003) (FACTA) [U.S. Government 2003], Gramm-Leach Bliley [U.S. Government 1999], and The Sarbanes-Oxley Act [U.S. Government 2002]. Once sensitive data is no longer needed, it should be permanently removed from storage.

Although wary users may diligently empty their trash bins and even reformat their storage, they may be unaware that typical deletion methods only make files invisible to users, while leaving behind recoverable sensitive data. Typical file deletion only updates the file's metadata (e.g., pointers to the data) for bookkeeping purposes, while often leaving the file data intact. Even recreating the file system does not ensure secure deletion. For example, the MSDOS `format` command has been shown to only overwrite 0.1% of the data [Garfinkel and Shelat 2003].

A remedy is secure deletion, which should render data irrecoverable. Many implementations of secure deletion use partition- or device-wide encryption or overwriting techniques. However, these coarse-grained methods are cumbersome to use when only a few files need to be securely deleted. Furthermore, securely deleting an entire device or partition may be infeasible for embedded devices.

In contrast, per-file secure deletion is concerned with securely removing a specific file's content *and* the metadata describing the file (e.g., name, ownership, last access time). The advantages of deleting files in a truly secure manner are numerous. For example, it can assist with implementing privacy policies concerning the selective destruction of expired data (e.g., client files), complying with government regulations to dispose of sensitive data, deleting temporarily shared trade secrets, military applications demanding immediate destruction of selected data, and disposing of media in one-time-use applications. All of this can be done without turning off the computer or disrupting access to storage services.

We introduce TrueErase [Diesburg et al. 2012], a framework that irrevocably deletes data and metadata upon user request. TrueErase goes to the heart of the user's mental model: securely deleted data, like a shredded document, should be irrecoverable. Our contributions are as follows:

—Design and implementation of a holistic, per-file secure-deletion framework that works alongside the legacy storage data path.
—Verification of the core framework component through black-box permutation of inputs and two-version programming.
—Evaluation of our framework on legacy hard drives.

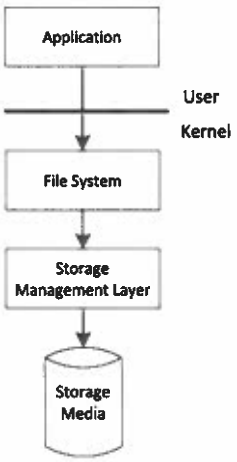

Fig. 1. The legacy storage data path.

4 cont.

—Evaluation of our framework on NAND flash through an expansion of common NAND-management software.
—An in-depth analysis of the challenges of implementing and coordinating secure deletion throughout the legacy storage data path.

The remainder of this article is organized as follows: Section 2 gives background on the operating-system storage data path, compares existing approaches, and discusses applicable threat models. Section 3 introduces an overview of the TrueErase solution. Section 4 gives the design of TrueErase. Section 5 discusses implementation of TrueErase. Section 6 addresses verification of TrueErase, and Section 7 evaluates TrueErase's performance. Section 8 relates TrueErase to existing systems, Section 9 provides discussion and future work, and Section 10 concludes.

2. BACKGROUND AND EXISTING APPROACHES

The legacy OS storage data path hinders secure deletion. To illustrate, we provide background on the storage data path and the limitations it imposes on existing secure-deletion solutions. Next, we compare existing approaches through a discussion of the desirable characteristics of secure deletion, followed by the applicable adversarial and threat models.

2.1. Storage Data Path Background and Challenges

5

Storage requests travel from user applications to the storage device through the legacy operating-system storage data path, which can be thought of in layers (Figure 1). Applications make storage requests through system calls; the requests cross the kernel boundary and then are handled by a file-system layer. The file-system layer often contains a generic layer such as the Virtual File System (VFS) Layer [Kleiman 1986] in Linux, which provides common functions and allows different file systems to co-exist (e.g., local file systems such as ext2/3/4 and remote file systems such as NFS).

After the file system has finished processing a request, the requests are handled by a storage-management layer (often referred to as a *block layer*). This layer transforms the requests into storage-specific granularities. The storage-management layer contains a unified storage API and device-specific low-level drivers to handle the vendor-specific representations of structures on the storage media.

Finally, the requests are passed to the storage medium itself (e.g., HDD or SSD). Increasingly, storage devices contain additional logic to optimize, cache, or otherwise handle storage requests before the data is handled by the raw storage. The operating system may or may not be able to control this logic to various degrees. For example, it is usually not possible to read all the raw storage locations on many flash devices [King and Vidas 2011; Wei et al. 2011; Bonetti et al. 2013].

The legacy storage data path imposes the following challenges in designing a secure-deletion solution:

—*No Interlayer Coordination.* Many secure-deletion solutions are implemented inside one layer of the legacy storage data path. Consequently, layers that are unaware of the secure-deletion functionality might not honor secure-deletion requests. For example, a secure deletion issued by a program might not delete sensitive data from file systems that use a journal [Shred 2012]. File-system- and application-layer solutions are generally unaware of the storage medium and have limited control over the physical location of data and metadata. The storage-management layer has no information about a block's file ownership [Ganger 2001; Sivathanu et al. 2004] and cannot support per-file deletion. In addition, storage such as NAND flash may leave behind multiple versions of sensitive data. To illustrate, a secure deletion issued by a program might not be honored by optimization software used on typical flash devices that keep versions of the data [Wei et al. 2011]. Thus, enforcing secure deletion requires a solution that spans the entire data path.

—*No Pre-existing Deletion Operation.* Read and write operations are routinely sent from file systems to the storage device. However, to optimize performance, file systems generally do not issue requests to erase file content. Instead, the file system sends a series of write requests to remove references to data blocks and set the file size and allocation bits to zero.

—*Complex Storage-Data-Path Optimizations.* Secure deletion needs to retrofit legacy asynchronous optimizations. In particular, storage requests may be reordered, concatenated, split, consolidated (applying one update instead of many to the same location), cancelled, or buffered while in transit.

—*Lack of Data-Path-Wide Identification.* Tracking sensitive data throughout the operating system is complicated by the possible reuse of data structures, ID numbers, and memory addresses.

—*Verification Challenges.* Although verification is often overlooked by various solutions, we need to ensure that (1) secure deletions are correctly propagated throughout the storage data path, and (2) assumptions are checked whenever possible.

As a result, it is difficult to add secure-deletion functionality to the legacy storage data path in a backwards-compatible way. All of the above challenges must be overcome to achieve a working, verifiable secure-deletion solution.

One approach is to redesign all the data path components and interfaces from a clean slate. Unfortunately, doing so would require extensive changes to all storage drivers, file systems, and less visible components such as storage schedulers. Such a change would require industry acceptance and vast amounts of time and money.

A more practical approach is to design a method to allow backward-compatible changes toward secure deletion in a way that does not break the current storage data path components or interfaces, or else only modifies the interface where absolutely necessary. We embrace this pragmatism in the form of designing and adding a second, parallel data path that explicitly propagates secure-deletion information through all the storage layers.

2.2. Desirable Characteristics of Secure Deletion Systems

We distinguish the need for TrueErase from existing approaches by enumerating desirable characteristics of secure deletion systems.

—*Per-File.* Fine-grained secure deletion brings greater control to the user while potentially reducing costly secure deletion operations on the storage device.
—*No Assumed Secure-Deletion Storage Medium.* Encryption systems securely delete by destroying the keys of encrypted files. However, these systems are complex due to managing per-file keys, providing efficient random access, data and metadata padding and alignments, etc. If encryption keys are stored on persistent storage such as a HDD or SSD, deletion or changing of the encryption keys may run into the same problems as regular file deletion through the storage data path as explained in Section 2.1. Encryption secure deletion systems may either rely on a special-purpose secure-deletion medium to store keys or else not address the specifics of key erasure on media if the system is a remote secure deletion system. A framework such as TrueErase can work in conjunction with such systems to provide secure deletion of encryption keys. More discussion can be found in Section 9.3.
—*Storage-Medium-Agnostic.* A secure-deletion framework should be general enough to accommodate traditional hard drives, solid-state storage such as NAND flash and emerging technologies, and should work on portable devices that use either type of storage.
—*Limited Changes to Legacy Code.* Per-file secure deletion requires some information from the file system. However, getting such information should not involve modifying thousands of lines of legacy code.
—*Metadata Secure Deletion.* Secure deletion should also remove metadata, such as the file name, size, etc.
—*Crash Handling.* A secure-deletion solution should be consistent and have reasonable recovery semantics in the face of system crashes.

Table I summarizes how existing solutions fare with respect to the characteristics listed above and are organized by data path layer. TrueErase meets all the listed positive characteristics. Also note that TrueErase is the only solution to be storage-medium-agnostic and handle the nuances of secure deletion on local storage. See Section 8 for a more detailed description of related work.

Adversarial model. We assume the attacker has access to all software (e.g., user applications and operating system) and the disk image after a secure deletion or clean system recovery (in case of crash) and should not be able to recover any data or metadata associated with a securely deleted file. Ideally, our system should be robust against an attacker that has access to both software and hardware (local storage). However, due to the remapping performed by the storage device, it is possible (but very difficult) for an attacker to gain information from internal bad sector or block lists (see Section 9.1). However, when combined with encryption, our system could be completely robust against an attacker with hardware access (see Section 9.1).

Trusted Computing Base. Our research question focuses on a simplified scenario: under benign user control and system environments, what holistic solution can we design and build to ensure that secure deletion of a file is honored throughout the storage data path? Our trusted computing base consists of an environment where we have the full control of the entire storage data path in an uncompromised, single-user, single-file-system, non-RAID, nondistributed deployment, with swapping disabled. However, we do not foresee limitations beyond additional verification efforts for our framework to support a machine with multiple users running multiple file systems and RAIDs.

Table I. Existing Secure-Deletion Methods and Characteristics

Existing secure-deletion methods and characteristics	C: handle crashes M: securely delete metadata L: limited changes to legacy code S: storage-medium-agnostic D: no assumed secure-deletion storage medium F: per-file						
Storage Layer	Solution	F	D	S	L	M	C
User-space	User-space solution on top of flash file system [Reardon et al. 2011]	✓	✓		✓	?	
	Overwriting tools [Apple, Inc. 2012; Garlick 2012; Gauthier and Jagdmann 2012; Shred 2012; Vier 2012]	✓	✓		✓		
File system	Modified flash file systems – device erasures and/or overwriting [Reardon et al. 2011; Sun et al. 2008]	✓	✓		✓	?	?
	Modified flash file systems – encryption with key erasure [Lee et al. 2008]	✓	✓		?	✓	?
	Data Node Encrypted File System [Reardon et al. 2012]	✓	✓		✓	✓	✓
	Stackable file system deletion [Joukov et al. 2006; Joukov and Zadok 2005]	✓	✓		✓	✓	✓
	Modified file system – deletion through overwriting [Bauer and Priyantha 2001; Joukov et al. 2006; Joukov and Zadok 2005]	✓	✓		✓	✓	✓
	Modified file system – deletion through encryption [Peterson et al. 2005]	✓	✓		?		?
Storage-management	Secure delete encrypted device/partition key [Hughes 2002; Apple, Inc. 2012; Imation 2015; Kingston 2012; Thibadeau 2006]		✓		✓	✓	✓
	Specialized hard drive commands [Hughes 2004]		✓		✓	✓	✓
	Specialized flash medium commands (page granularity) [Wei et al. 2011]	✓	✓		✓		
Remote/special secure-deletion medium	Dedicated server(s) for encryption keys [Geambasu et al. 2009; Perlman 2005]	✓		✓	✓	?	✓
	Separate securely-deleting medium to delete encrypted data stored elsewhere [Reardon et al. 2013b; Crescenzo et al. 1999]	✓		✓	✓	✓	?
	Encrypted backup system [Boneh and Lipton 1996]	✓		✓	✓		?
Data-path-wide	Semantically-Smart Disk Systems [Sivathanu et al. 2004; Sivathanu et al. 2003]	✓	✓		✓		✓
	Type-Safe Disks [Sivathanu et al. 2006]	✓	✓				✓
	TrueErase	✓	✓	✓	✓	✓	✓

Note: The boxes that are marked with ✓ indicate the solution meets the corresponding desired characteristic. The boxes that are marked with ? indicate that the publication does not clearly explain whether it meets the corresponding characteristic.

Section 9.3 discusses these possible expansions, as well as the additions necessary for TrueErase to work with swap, log-structured and versioning file system, and with emerging storage media.

3. SOLUTION OVERVIEW

TrueErase (Figure 2) is a holistic framework that propagates file-level information to the block-level storage-management layer via an auxiliary communication path, so that per-file secure deletion can be honored throughout the data path. Components along the storage data path can be made to interact with our auxiliary data path to report and query secure-deletion information while preserving legacy behavior. We later show that the cost of secure deletion using this auxiliary secure storage data path is comparable to encryption-free alternatives with similar deletion mechanisms.

TrueErase consists of four main components: (1) a user-space API to specify which files or directories are to be securely deleted, (2) tracking and propagating information

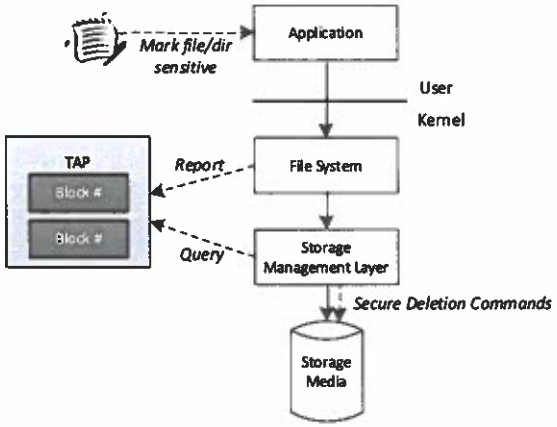

Fig. 2. TrueErase framework. *Note:* New additions represent our augmented data path.

7 cont:

across storage layers, (3) enforcing secure deletion via storage-specific mechanisms, and (4) exploiting file-system consistency properties for verification.

8.

TrueErase allows a user to mark individual files as *sensitive*, so that they will be deleted securely. If a file is marked as sensitive at deletion time, then the sensitive file content, metadata, and associated copies within the storage data path will be deleted securely. In the case of a sensitive folder, all files underneath will be securely deleted. A legacy application can then issue normal deletion operations on sensitive files and folders, and the operations will be carried out securely.

The file system generates information about pending write and deletion events. This information is sent along the legacy storage data path as normal. However, the file system can *also* report this information, along with contextual information about the data, to a centralized module for later retrieval. We add this centralized module and call it *Type-Attribute Propagation*, or TAP. Contextual information can include the sensitivity status of the block, the type of block (e.g., data, metadata, extended attribute, etc.), and a unique tracking ID.

Once a write or deletion event occurs at the storage management layer through the legacy storage data path, the storage management layer can query the TAP module for additional information about the block it is ready to write. This layer of the storage data path is responsive to the type of underlying storage, and if necessary, it can choose a secure deletion method suitable for the specific storage type.

We further rely on the presence of basic file-system-consistency properties [Sivathanu et al. 2005], which have been shown to be a requirement of secure deletion. These basic properties can be found in modern file systems; however, since Sivathanu et al. [2005] have verified the properties in ext3, we chose to implement our prototype in ext3. In addition to preserving consistency during crash scenarios, the file-system-consistency properties allow us to enumerate file-system behavior during deletion operations for verification purposes. Further, all file-system update events are reported to our framework to verify that we are tracking all important events.

Finally, our solution for NAND flash storage relies on an extension and modification to common NAND-flash-management software called the *flash translation layer* (FTL). This software may either be an operating system driver (e.g., in embedded devices) or built into the device (e.g., most thumb drives, solid-state drives, and SD cards). Our prototype expands the software as an operating system driver, but it is not unreasonable for this functionality to be added to hardware. New hardware functionality in the form of the ATA TRIM command [Shu and Obr 2007], which allows a SSD to discard blocks

no longer in use by the file system, has recently been added to the SSD hardware interface; this functionality has similar properties to our proposed expansion and is discussed further in Section 9.2. This is similar to other designs that introduce new or enhanced NAND FTL functionality [Liu et al. 2012; Wu and He 2012; Lu et al. 2013; Jimenez et al. 2014; Jeong et al. 2014] or propose manufacturing changes to the FTL [Zhang et al. 2012; Zhao et al. 2013].

Some proposed or deployed file systems bypass the FTL entirely and directly control raw NAND flash via device-layer commands [Wu and Zwaenepoel 1994; Woodhouse 2001, 2008; Engel and Mertens 2005; Lim and Park 2006; UBIFS 2008; Josephson et al. 2010; Kim et al. 2012; Manning 2012]. However, a design goal of TrueErase is to work on a variety of system configurations without an assumed type of storage. Thus, TrueErase is not dependent on a NAND-flash-specific file system.

Finally, designs based on changing (encrypting or transforming) the *contents* of the data [Rivest 1997; Peterson et al. 2005] cannot perform secure deletion successfully if the encryption key or transform cannot be verified to be securely deleted. Thus, TrueErase tracks sensitive data locations all the way down to the storage device itself to verify erasure.

4. TRUEERASE DESIGN

We simplified the TrueErase framework design through the following observations and formulated guidelines:

—We leave the legacy data path request ordering intact to simplify verification and to keep legacy semantics intact.
—Per-file secure deletion is a performance optimization over applying secure deletion to all file removals. Thus, we can simplify tricky decisions about whether a case should be handled securely by handling it securely by default.
—Persistent states complicate system design with mechanisms to survive reboots and crashes. Thus, our solution minimizes the use of persistent states.

The major areas of TrueErase's design include (1) a user-space API to specify which files or directories are to be securely deleted, (2) tracking and propagating information across storage layers via a centralized module named *TAP*, (3) enforcing secure deletion via general and storage-specific mechanisms added to the storage-management layer, and (4) exploiting file-system consistency properties to enumerate cases for verification. We conclude this section with a discussion of corner cases and design choices in which we applied our observations and formulated guidelines discussed above.

4.1. User-Space API

TrueErase is designed for scenarios in which some files can be considered sensitive and others are nonsensitive. A reasonable dividing line would be to classify operating system and program files as nonsensitive, leaving user-created files as sensitive (such as a user's home directory in Linux). For example, all files under the /home directory could be classified as sensitive automatically in UNIX-like operating systems [Russell et al. 1994]. A user who wishes to have even more control can follow traditional permission semantics and apply common attribute-setting tools to mark a file or directory as *sensitive*, which means the sensitive file, or all files under the sensitive directory, will be securely deleted when removed. A legacy application can then issue normal deletion operations, which are carried out securely, to remove sensitive file and directory data content, metadata, and associated copies within the storage data path. Other methods for handling and marking sensitive files could include the use of a security token attached to the disk controller [Butler et al. 2008] or policies that tie files to certain applications [Enck et al. 2008].

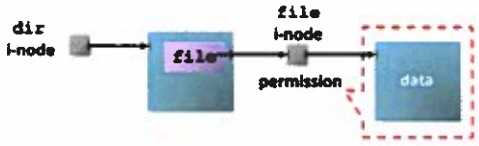

Fig. 3. Traditional Permissions.

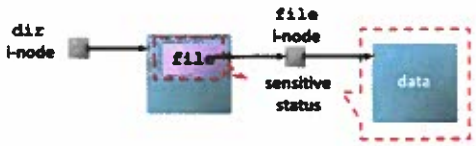

Fig. 4. TrueErase Secure Deletion Sensitive Status Model.

TrueErase uses traditional permission semantics to set files and directories as sensitive using file and directory attributes. However, there are certain deviations from this traditional permission model.

Toggling the Sensitive Status: Before the status of a file or directory is toggled from nonsensitive to sensitive, older versions of the data and metadata may have already been left behind. Without tracking all file versions or removing all old versions for all files, TrueErase can enforce secure deletion only for files or directories that have remained sensitive since their creation. Should a nonsensitive file be marked sensitive, secure deletion will be carried out only for the versions of the metadata and file content created after that point.

Name Handling. Another semantic deviation occurs when securely deleting a file name. Each file and directory has a corresponding piece of metadata that uniquely identifies it and is used for permissions. In the Unix world, this metadata is called an i-node. A directory is typically represented as a special file which contains file names and other metadata information as data content. Although the permissions of a file are applied to its data content through its i-node, permission to handle its name is controlled by its parent directory's i-node (Figure 3). Thus, should we mark the parent directory as sensitive, we also need to modify the parent directory's parent directory to make its name sensitive. The process would continue until the root directory, marking all files sensitive, contradicting our goal of achieving per-file secure deletion.

Instead, under TrueErase, the sensitive status of a file is applied to its name as well (Figure 4). Thus, marking a file sensitive will also cause some of its parent directory's data content to be securely handled, even if the parent directory itself is not marked sensitive.

Links. Similar to legacy semantics, a secure deletion of a hard link is performed when the link count decreases to zero. Symbolic link names and associated metadata should be securely deleted, and file data written to a symbolic link will be treated sensitively if the link's target file is sensitive.

4.2. File-System-Layer Information Tracking and Propagation

The file system generates requests and information related to deletion that must be tracked. Tracking and propagating secure-deletion information from the file system to the lower layers is done through a centralized type/attribute propagation (TAP) module.

4.2.1. Data Structures and Globally Unique IDs. All TAP *tracking records* directly correspond to write requests from the file system. The records that correspond to block updates are called *write entries*, and the records that correspond to deletions are called *deletion reminders*. Write entries can be associated with deletion reminders. For example, a truncate operation involves deleting some data blocks and updating the file size information stored in the metadata. We can link them together and propagate both along the data path.

Table II. The TAP Interface

Description	Use	Where
`TE_report_write(GUID, block type, secure status)`: Creates or updates a TAP write entry associated with the GUID.	Reports file system writes.	In journaling code and file system code; wherever a block is submitted for writing.
`TE_report_delete(metadata GUID, metadata block type, metadata secure status, deletion sector number)`: Creates or updates a TAP deletion reminder that contains the sector number. The reminder is attached to the write entry (created as needed) associated with the metadata GUID.	Reports file system deletions.	File system upon delete. Also in journaling code to delete journal blocks after they are no longer needed.
`TE_report_copy(source GUID, destination GUID, flag)`: Copies information from a write entry corresponding to the source GUID to a new entry corresponding to the destination GUID. The flag determines whether deletion reminders should be transferred.	Reports data-copying events (e.g., often used in journaling).	Journaling code; wherever writes for the journal are submitted.
`TE_check_info(GUID)`: Used to query TAP to retrieve information about a block layer write with a specific GUID.	Queries information.	Storage-management drivers and file system.
`TE_cleanup_write(GUID)`: Removes the write entry with a specified GUID. This call also handles the case in which a file has already been created, written, and deleted before being flushed to storage.	Cleanup after information is no longer needed.	Storage-management drivers and file system.

Since data structures (e.g., block I/O structures), namespaces (e.g., i-node numbers), and memory addresses can be reused, correct pairing of write entries and deletion reminders requires a unique ID scheme. In particular, the IDs need to be (1) accessible throughout the data path and (2) altered when their content association changes. TrueErase embeds a page ID (globally unique) in each memory page structure, which is accessible throughout the data path. The ID is set at page allocation time, so that the same page reallocated to hold versions of the same logical block has different IDs. We choose to put unique IDs in the page structures, as opposed to other more transient data path structures such as requests, because the page is directly associated with the data or metadata content from the moment the content is created and stays associated with the content until the content is freed.

Many different tracking granularities exist in the data path, such as logical blocks for file systems, concatenated requests, physical sectors, and device-specific units. TAP tracks by physical sector, because the physical sector is unique to the storage device and can be computed from anywhere in the storage data path. The page ID and the physical sector number obtained by TAP from lower layers form a *global unique ID (GUID)*, which we can use to uniquely associate information stored in TAP tracking records with the smallest applicable secure-deletion unit in the storage data path.

In addition to the GUID, TAP write entries can contain more information about the update. At minimum, write entries must contain the update's type (e.g., i-node and data) and the security status (sensitive or not sensitive). TAP write entries also contain an optional link to a TAP deletion reminder, which contains a list of physical sector numbers of blocks to be deleted and a field to specify that the deletion operation should happen either before or after the sector is written.

4.2.2. TAP Interface and Event Reporting. Through the TAP interface (Table II), the file system must report all important events that might modify or delete sensitive information: file deletions and truncations, file updates, and certain journaling activities.

File Deletions and Truncations. Data-block deletion depends on the update of certain metadata, such as free-block bitmaps. TAP allows file systems to associate data-block (actually data-sector) deletion events with metadata update events via TE_report_delete(). Within TAP, write entries are associated with respective deletion reminders and their deletion lists.

Eventually, the file system flushes the metadata update. Once the block-layer interface receives a sector to write, the interface uses the GUID to look up information in TAP through TE_check_info(). If the write entry is linked to a deletion reminder, the storage-management layer must securely delete the sectors on the deletion list before writing the metadata update.

Additionally, the storage-management layer can choose a secure-deletion method that matches the underlying medium. For NAND flash, triggering the NAND-specific erase command can erase data (details in Section 5.3). For a hard drive, the sectors can be directly overwritten with random data.

After secure deletion has been performed and the metadata has been written to storage, the block-layer interface can call TE_cleanup_write() to remove the associated TAP write entries and deletion reminders.

For data-like metadata blocks (e.g., directory content), deletion handling is the same as that of data blocks.

File Updates. Performing deletion operations is not enough to ensure all sensitive information is securely deleted. By the time a secure-deletion operation is issued for a file, several versions of its blocks might have been created and stored (e.g., due to flash optimizations or file system journaling), and the current metadata might not reference old versions. One approach is to track all versions so that they can be deleted at secure-deletion time. However, tracking these versions requires persistent states, and thus recovery mechanisms to allow those states to survive failures.

TrueErase avoids persistent states by tracking and deleting old versions along the way. That is, secure deletion is applied in-place to an overwritten sensitive block before each update is written to the same location (*secure write* for short). Therefore, in addition to deletion operations, TrueErase needs to track all in-transit updates of sensitive blocks.

The file system can report pending writes to TAP through TE_report_write(). Eventually, the block-layer interface will receive a sector to write and can then look up information via TE_check_info(). If the corresponding write entry states that the sector is secure, the storage-management layer will write it securely. Otherwise, it will be written normally. After the sector is written, the block-layer interface calls TE_cleanup_write().

Journaling and Other Events. Sometimes file blocks are copied to other memory locations due to file system consistency, performance, and accessibility reasons (e.g., file-system journal and bounce buffers). When that happens, any write entries associated with the original memory location must be copied and associated with the new memory location. If there are associated deletion reminders, whether they are transferred to the new copy is file-system-specific. For example, in ext3 (a journaling file system), the deletion reminders are transferred to the memory copy in the case of bounce buffers because the original write entry never gets written. Memory copies are reported to TAP through TE_report_copy().

Crash Handling. TAP contains no persistent states and requires no additional recovery mechanisms. Persistent states stored as file-system attributes are protected by the file system journal-recovery mechanisms. At recovery time, a journal is replayed with all operations handled securely, even if files or directories are not marked sensitive. We then securely delete the entire journal. To hunt down leftover sensitive data blocks, we

apply the method used by Bauer and Priyantha [2001]. Specifically, during recovery we identify and securely delete data blocks that are not yet pointed to by any i-node.

4.3. Enhanced Storage-Management Layer

TrueErase does not choose a secure-deletion mechanism until a storage request has reached the storage-management layer. By doing so, TrueErase ensures that the chosen mechanism matches the characteristics of the underlying storage medium. In addition, TrueErase can carry out different secure-deletion methods in accordance with different requirements and government regulations [U.S. Department of Defense 1995; U.S. NIST 2006]. To illustrate the flexibility of the TrueErase framework to handle different storage media, we include our design for both NAND flash storage and hard drives. We added two generic secure-deletion commands, used for various storage media.

Secure_delete(locations). This call irrevocably erases information in the specified locations using drive-specific techniques.

Secure_write(locations, data). This call is made when new secure data will potentially be overwriting old secure data. Secure_delete() will be called on the specified storage locations to erase the old data before overwriting it with the new. This call is especially relevant to NAND flash, which may otherwise leave behind copies of the overwritten data.

Device-specific implementation details of these two commands can be found in Section 5.3.

4.4. File-System-Consistency Properties

We cannot guarantee the correctness of our framework if it interacts with file systems with arbitrary behaviors. Therefore, we applied the consistency properties of file systems defined in Sivathanu et al. [2005] to enumerate corner cases. (These properties also rule out file systems such as ext2 and FAT, which are prone to inconsistencies due to crashes.) By working with file systems that adhere to these properties, we can simplify corner-case handling and verify our framework systematically.

In a simplified sense, as long as pieces of file metadata reference the correct data and metadata versions throughout the data path, the system is considered consistent. In particular, we are interested in three properties in Sivathanu et al. [2005]. The first two are for nonjournaling-based file systems. In a file system that does not maintain both properties, a nonsensitive file may end up with data blocks from a sensitive file after a crash recovery. The last property is needed only for journaling-based file systems.

—*The reuse-ordering property* ensures that once a file's block is freed, the block will not be reused by another file before its free status becomes persistent. Otherwise, a crash may lead to a file's metadata pointing to the wrong file's content. Thus, before the free status of a block becomes persistent, the block will not be reused by another file or changed into a different block type. With this property, we do not need to worry about the possibility of dynamic file ownership and types for in-transit blocks.

—*The pointer-ordering property* ensures that a referenced data block in memory will become persistent before the metadata block in memory that references the data block. With this ordering reversed, a system crash could cause the persistent metadata block to point to a persistent data block location not yet written. This property does not specify the fate of updated data blocks in memory once references to the blocks are removed. However, the legacy memory page cache prohibits unreferenced data blocks from being written. The pointer-ordering property further indicates that right after a newly allocated sensitive data block becomes persistent, a crash at this

point may result in the block being unreferenced by its file. To cover this case, we perform secure deletion on unreferenced sensitive blocks at recovery time.

—*The nonrollback property* ensures that older data or metadata versions will not overwrite newer versions persistently—critical for journaling file systems with multiple versions of requests in transit. That is, we do not need to worry that an update to a block and its subsequent secure deletion will be written persistently in the wrong order.

With these consistency properties, we can identify the structure of secure-deletion cases and handle them by: (1) ensuring that a secure deletion occurs before a block is persistently declared free, (2) the dual case of hunting down the persistent sensitive blocks left behind after a crash but before they are persistently referenced by file-system metadata, (3) making sure that secure deletion is not applied (in a sense, too late) to the wrong file, (4) the dual case of making sure that a secure block deletion is not performed too early and gets overwritten by a buffer update from a deleted file, and (5) handling in-transit versions of a storage request (mode changing, reordering, consolidation, merging, and splitting). Buffering, asynchrony, and cancelling of requests are handled by TAP.

4.5. Other Design Points

Consolidation of Requests. Consolidation of requests cannot be allowed in certain circumstances (e.g., a secure overwrite might disappear when consolidated with a nonsecure overwrite). In this case, TrueErase needs to disable storage-built-in write caches or use barriers and device-specific flush calls to ensure that persistent updates are achieved [Nightingale et al. 2008]. When consolidation is permitted (such as in the page cache or journal), TrueErase interprets an update's sensitive status during consolidation conservatively. As long as one of the updates to a given location is sensitive, the resulting update will be sensitive.

Security Status for Shared Blocks. A block often can hold metadata for many files, likely with mixed sensitive status. Thus, updating nonsensitive metadata may also cause the sensitive metadata stored on the same block to appear in the storage data path. TrueErase handles this case by treating a shared metadata block as sensitive as long as one of its metadata entries is sensitive.

Partial Secure Deletion of a Metadata Sector. Securely deleting a file's metadata only at the storage level is insufficient, since a metadata block shared by many files may still linger in the memory while containing the sensitive metadata. If the block is written again, the metadata thought to be securely deleted will return to storage. In these cases, TrueErase zeros out the sensitive metadata within the block in memory, and then updates the metadata as a sensitive write.

5. TRUEERASE IMPLEMENTATION

We prototyped the TrueErase framework under Linux 2.6.25.6. We applied our framework to the ext3 journaling file system version 1.41.3 [Tweedie 1998] due to its adherence to file-system-consistency properties [Sivathanu et al. 2005]. We implemented TrueErase to work on both a NAND flash chip and a traditional hard drive. Overall, our user-space API required 198 lines of C code; TAP, 939; secure-deletion commands for flash, 592; secure-deletion commands for hard drives, 378; and a verification framework, 8,578. Our actual file system changes were minimal; we inserted around 60 TAP-reporting calls in ext3 and jbd, which between them have about 15,000 lines of legacy code.

This section discusses the TrueErase implementation concerning the user model, file system interactions with the TAP module, and the extended storage-management layer for secure deletion commands.

5.1. User-Space API

A user can execute the legacy `chattr +s` command to mark a file or directory as sensitive. However, by the time a user can set attributes on a file, its name may already be stored as nonsensitive. The problem is that the actions of creating a file and setting its sensitive attribute are not performed atomically. Without modifying the OS, one remedy is to cause files or directories under a sensitive directory to inherit a secure-deletion attribute when they are created.

As a more convenient interface, we also provide end users with a set of wrapper scripts. Each (1) creates an empty file or directory with a pseudoname, (2) marks it sensitive, and (3) renames it to the sensitive name. We name these scripts `smkdir` and `screat`, which create secure directories and files, respectively. The file or directory will be treated as sensitive after it is marked as such, so these scripts might allow an untracked flush to storage of the pseudo name and the corresponding i-node linked to the pseudo name that contains information about the file or directory (i.e., size of zero bytes, ownership, and creation time).

Currently, `chattr` does not support symbolic link names and associated metadata. However, file data written to a symbolic link will be treated sensitively if the link's target file is sensitive.

5.2. TAP Module

TAP is implemented as a kernel module. We give a brief background on ext3 to clarify its interactions with TAP.

5.2.1. File System Background. Ext3 is a journaling file system based on the nonjournaling ext2. It performs journaling by sending pending blocks to be written to the jbd (journaling block device) driver, which controls when and how blocks are written to the storage device.

File Truncation / Deletion. Ext3 deletes the data content of a file via its truncate function, which involves updating ***(t1)*** the i-node to set the file size to zero, ***(t2)*** metadata blocks to remove pointers to data blocks, and ***(t3)*** the bitmap allocation blocks to free up blocks for reuse.

Deleting a file involves removing the name and i-node reference from the directory, adding the removed i-node to an orphan list; truncating the entire file via (t1) to (t3); removing the i-node from the orphan list; and updating the i-node map to free the i-node.

Journaling: Typical journaling employs the notion of a transaction, so the effect of an entire group of updates is either all or nothing. With group-commit semantics, the exact ordering of updates within a transaction may be relaxed while preserving correctness, even in the face of crashes. To achieve this effect, all writes within a transaction are ***(j1)*** journaled or committed to storage persistently (*commit stage*); then ***(j2)*** propagated to their final storage destinations (*checkpoint stage*), after which ***(j3)*** they can be discarded from the journal.

A committed transaction is considered permanent even before its propagation. Thus, once a block is committed to be free, it can be used by another file. At recovery time, committed transactions in the journal are replayed to repropagate or continue propagating the changes to their final destinations. Uncommitted transactions are aborted.

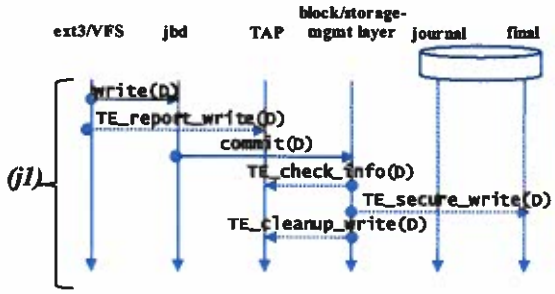

Fig. 5. Secure Data Updates. D is the data block in stages of being securely written. Dotted arrows show additional TrueErase calls being invoked to achieve secure data updates.

Jbd. Jbd differentiates data and metadata. We chose the popular ordered mode, which journals only metadata but requires data blocks to be propagated to their final destination before the corresponding metadata blocks are committed to the journal.

5.2.2. Deployment Model. All truncation, file deletion, and journaling operations can be expressed and performed as secure writes and deletions to data and metadata blocks. The resulting deployment model and its applicability are similar to those of a journaling layer. We inserted around 60 TAP-reporting calls in ext3 and jbd, with most collocated with block-layer interface write submission functions and dirty (marking as modified) functions (e.g., `ext3_journal_dirty_data`).

Applicable Block Types. Secure writes and deletions are performed for sensitive data, i-node, extended-attribute, indirect, and directory blocks, as well as corresponding structures written to the journal. Remaining metadata blocks (e.g., superblocks) are frequently updated and shared among files (e.g., bitmaps) and do not contain significant information about files. By not treating these blocks as sensitive, we reduce the number of secure-deletion operations.

Secure data updates (Figure 5): Ext3 calls `TE_report_write()` on sensitive data block updates, and TAP creates per-sector write entries. Updates to the same TAP write entries with the same GUID are consolidated; this behavior reflects that of the page cache.

The data update eventually reaches the block-layer interface (via commit), which retrieves the sensitive status via `TE_check_info()`. The layer can then perform the secure-write operation and invoke `TE_cleanup_write()` to remove the corresponding write entries.

Secure metadata updates (Figure 6): A metadata block must be securely written to and deleted from the journal.. Ext3 reports metadata updates to TAP via `TE_report_write()`. Repeated updates to the same metadata entries will lead to updates to TAP write entries with matching GUIDs. To commit a transaction, jbd calls `TE_report_copy()` to clone write entries and their sensitive status, with the storage destination retargeted to persistent journal locations (instead of the final storage destination). When the storage-management layer receives the journal commit update request, it performs checking, invokes the secure-write method, and cleans up the cloned journal write entries via `TE_cleanup_write()`, such as shown in the ***(j1)*** phase of Figure 6.

The original write entries in TAP remain until they are propagated to their final storage destinations via a journal checkpoint. The storage-management layer performs checking, securely writes the metadata to the final destination, and cleans up the corresponding write entries, as shown in the ***(j2)*** phase of Figure 6.

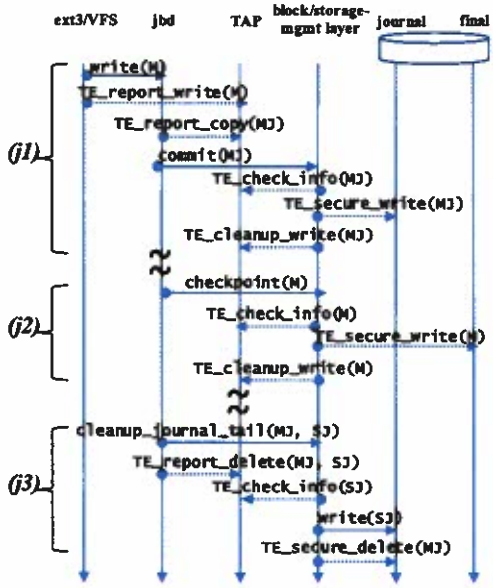

Fig. 6. Secure metadata updates. M is the reported metadata block; MJ is the reported metadata journal block; and SJ is the reported journal superblock. Dotted arrows show additional TrueErase calls being invoked to achieve secure metadata updates.

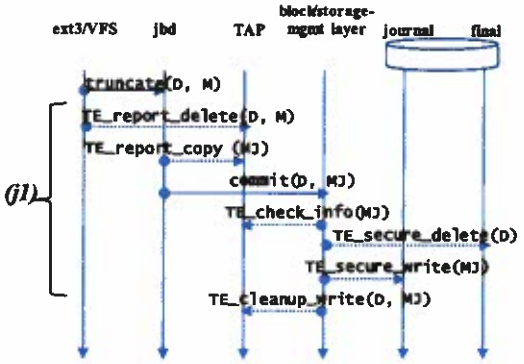

Fig. 7. Secure data deletion. M is the updated metadata block; D is the data block in various stages of secure deletion; MJ is the metadata journal block that corresponds to the updated metadata block M.

Jbd tracks in-use persistent journal locations through its own superblock. Periodically (or explicitly through an unmount), jbd checks the in-use status of journal locations in its log via `cleanup_journal_tail()`. If some of its log is no longer needed, it updates its superblock allocation pointers accordingly. We leverage this function to report journal locations no longer in use to TAP. If any of the locations match our previous sensitive metadata write entries, we securely delete the location after the updated journal superblock is written, as shown in phase *(j3)* of Figure 6. In the case of a crash, at file-system recovery time we securely delete all journal locations through `TE_report_delete()` once all committed updates have been securely applied (not shown in the figure).

Secure data deletions (Figure 7). When deleting sensitive file content, ext3's truncate function informs TAP of the deletion list and associated file i-node via

`TE_report_delete()`. Given the transactional semantics of a journal, we can associate the content-deletion event with the file's i-node update event instead of the free-block bitmap update event. Thus, we securely delete data before step ***(t1)***.

TAP will create and pair the i-node write entry with the corresponding secure-deletion reminder to hold the deletion list. When the write entry is copied via `TE_report_copy()`, reminders are transferred to the journal copy to ensure that secure deletions are applied to the matching instance of the i-node update.

When the storage-management layer receives the request to commit the update of the sensitive i-node to the journal, the interface calls `TE_check_info()` and retrieves the sensitive status of the i-node, along with the deletion list. The data areas are then securely deleted before the i-node update is securely written to the journal on storage.

Secure metadata deletions. During a file truncation or deletion, ext3 also deallocates extended attribute block(s) and indirect block(s). Those blocks are attached to the i-node's list of secure-deletion reminders as well.

To securely delete an i-node or a file name in a directory, the block containing the entry is securely updated and reported via `TE_report_write()`. Additionally, we need to zero out the i-node and variable-length file name in the in-memory copies, so that they will not negate the secure write performed at the storage-management layer.

If a directory is deleted, its content blocks will be deleted in the same way as the content from a file.

Miscellaneous Cases. Committed transactions might not be propagated instantly to their final locations. Across committed transactions, the same metadata entry (e.g., i-node) might have changed file ownership and sensitive status. Thus, jbd might consolidate, say, a nonsensitive update 1, sensitive update 2, and nonsensitive update 3 to the same location into a single nonsensitive update. As a remedy, once a write entry is marked sensitive, it remains sensitive until securely written.

5.3. Enhanced Storage Management Layer

This subsection discusses the implementation of secure-deletion commands in the storage-management layer for both NAND flash and a traditional hard drive. However, we first must provide some NAND flash background.

5.3.1. NAND Flash Background. NAND flash poses new challenges for secure deletion. Unlike hard drives, in-place overwrites are typically not allowed. Instead, once a location is written, it must be erased before it can be written again. Further, each storage location can be erased only 10K to 1M times [Cooke 2007], and many NAND flash drives have extra hidden physical locations that are used to hold additional information. For example, one article found between 6% and 25% more physical flash storage than the advertised logical capacities of several drives [Wei et al. 2011].

These physical challenges are commonly solved through the addition of a flash translation layer (FTL). The FTL sits atop the raw NAND flash interface (either in hardware or in software). For backward compatibility, the FTL exports a hard-drive interface that accepts only read and write requests. As a common optimization to mask slow writes and erases, when flash receives a request to overwrite a page, the FTL remaps the write to a pre-erased page, version-stamps it, and marks the old page as invalid, to be cleaned later during the garbage collection cycle. These invalid pages are not accessible to components above the block layer, but can be recovered by forensic techniques [Breeuwsma et al. 2009]. To prolong the lifespan of the flash, wear-leveling techniques are often used to spread the number of erasures evenly across all storage locations.

We can illustrate secure-deletion challenges with a common example. Suppose a typical hard-drive-oriented secure-deletion program issues overwrites to a file with

random bits. A typical flash-management layer would avoid the slow erases and spread the usage of flash locations by writing the random bits to another pre-erased area (if possible). The old pages are marked invalid or dead, but the dead data persist until garbage collection time, if they are even erased at all! Additionally, the FTL does not export any interface commands designed to immediately erase data selectively[1] or search for information on invalid pages. Thus, through the FTL interface, it is impossible to erase specific locations or discern if selective sensitive data persist on the medium.

Because raw flash devices without FTLs and their development environments are not widely available, we used SanDisk's DiskOnChip [SanDisk 2006] flash and the associated open source Inverse NAND File Translation Layer (INFTL) kernel module as our FTL. This allowed us access to the underlying raw flash blocks and pages. Although DiskOnChip is somewhat dated, our design is applicable to modern flash and development environments.

5.3.2. NAND Flash Extensions. We modified the existing Linux INFTL to incorporate secure deletion. INFTL uses a stack-based algorithm to remap logical pages to physical ones. INFTL remaps at the *flash block* level, where each 16Kbyte flash block contains 32 512-byte *flash pages*, with a 16-byte control area per page. A remapped page always has the same offset within a block.

A NAND page can be in three states: empty, valid with data, or invalid. An empty page can be written, but an invalid page has to be erased to become an empty one. We explain two pre-existing INFTL operations (write and read), and then we describe our two new operations (secure write and secure deletion). Finally, we discuss optimizations in our implementation.

INFTL Writes. INFTL uses a stack of flash blocks to provide the illusion of in-place updates. When a page P_1 is first written, an empty flash block $B_1\{\}$ is allocated to hold P_1 (i.e., $B_1\{P_1, -, -\ldots\}$. If P_i is written again (P_1'), another empty flash block $B_2\{\}$ is allocated and stacked on top of B_1, with the same page offset holding P1' (i.e., B2{P1', -, -...}. Suppose we write P_2, which is mapped to the same block. P_2 will be stored in B_2 because it is at the top of the stack (i.e., $B_2\{P_1', P_2, -, \ldots\}$ and its page at B_1's page offset for P_2 is empty {i.e., $B_1\{P_1, -, -, \ldots\}$.

The stack will grow until the device becomes full; it will then be flattened into one block containing only the latest pages to free up space for garbage collection.

INFTL Reads. For a read, INFTL traverses down the appropriate stack from the top and returns the first valid page. If the first valid page is marked deleted, or if no data are found, INFTL will return a page of zeros.

Secure Write. Our added secure write command is similar to the current INFTL in-place update. However, if a stack contains a sensitive page, we set its maximum depth to 1 (0 is the stack top). Once it reaches the maximum, the stack must be consolidated to depth 0. When consolidating, old blocks are immediately erased via the flash erase command, instead of being left behind.

Secure Deletion. Our added secure delete is a special case of a secure write. When a page is to be securely deleted, an empty flash block is allocated on top of the stack. All the valid pages, minus the page to be securely erased, are copied to the new block. The old block is then erased.

[1]The TRIM command is not meant for secure deletion and is further discussed in Section 9.2.

Optimizations. We aligned the flash block boundaries with logical file system block boundaries. This optimization is not TrueErase-dependent, so in our evaluation we performed logical block alignment on all kernels.

Locations listed in secure-deletion reminders tend to have high spatial locality. For our initial implementation, we only performed single-page secure deletions on flash blocks, which meant that a single block could incur multiple erasures while processing a single deletion reminder. We implemented an optimization that groups locations on the delete reminder list by flash block, and we use this information to securely delete multiple flash pages in a block with only one round of migration for other in-use pages.

Since the existing INFTL stack algorithm already tracks old versions, we implemented the delayed-deletion optimization for data blocks, which defers the secure-write consolidation to file deletion time. Basically, the maximum depth is no longer bounded. For metadata, delaying secure deletion is trickier and will be investigated in future work.

For correctness, because jbd does not allow reordering to violate file system constraints and our flash has no built-in cache, these optimizations do not affect the correctness of file system consistency semantics.

5.3.3. Hard-Drive Extensions. The Linux device mapper resides in the storage-management layer and serves to map (or stack) a logical block device on top of one or more physical devices. File systems may be directly mounted on the exported logical device. A simple device mapper target called "linear" maps a range of sectors onto a physical device. We based our TrueErase hard drive abstraction off the linear target; it (1) queries TAP for secure-deletion information related to a sector it receives for writing and (2) performs possible secure-deletion operations.

We explain our added functionality (secure deletion and secure write) below. Finally, we discuss implementation optimizations.

Secure Write: The goal of this functionality is to securely overwrite the sectors to be written with random data a specified number of times before performing the pending data or metadata write.

Suppose `TE_check_info()` indicates the current pending block I/O must be securely written. We first create a block filled with random bits directed to the pending block I/O sector address and send it down to the physical block device. We then must follow that submission by creating and submitting a *barrier* block I/O for reasons discussed below in the secure-deletion case. This procedure of processing the block I/O filled with random bits and barrier block I/O structure can be repeated for a desired number of overwrites. Finally, we send the current pending block I/O to the physical block device and call `TE_cleanup_write()`.

Secure Deletion: This functionality is called when `TE_check_info()` indicates that a list of sectors must be securely deleted either before or after the current pending metadata block I/O is processed. We first create a list of blocks filled with random bits, addressed to the sectors to be securely deleted, and send it down to the physical block device. Following this list, we create and submit a *barrier* block I/O, which (1) ensures block I/Os will not be reordered across the barrier in the Linux disk scheduler and (2) flushes the on-disk cache for disks that support barrier requests. This procedure of sending a list of erase-block I/Os and barrier is repeated for the desired number of secure-deletion overwrites. Finally, we send the current pending metadata block I/O to the physical block device and call `TE_cleanup_write()`.

Optimizations. If the user determines that a single hard-drive overwrite is sufficient to securely erase the previously written information, repeated secure overwrites may be

omitted. We chose to default to this case, given the practical impossibility of recovering singly overwritten data from a modern hard drive [Joukov et al. 2006; Wright et al. 2008].

6. VERIFICATION

We (1) tested the basic cases, assumptions, and corner cases discussed in Section 4.4 and (2) verified the state space of the TAP module.

6.1. Basic Cases

Sanity Checks: We verified common cases of secure writes and deletes for empty, small, and large files and directories using predetermined random file names and sector-aligned content. After deletion, we scanned the raw storage and found no remnants of the sensitive information. We also traced common behaviors involving sensitive and nonsensitive objects; when the operation included a source and a destination, we tested all four possible combinations. The operations checked included moving objects to new directories, replacing objects, and making and updating symbolic and hard links. We also tested sparse files. In all cases, we verified that the operations behaved as expected.

Emulated Workload: We ran the PostMark benchmark [Katcher 1997] with default settings, modified with 20% of the files marked sensitive, with random content. Afterwards, we found no remnants of sensitive information.

Missing Updates: To check that all update events and block types are reported, we looked for errors such as unanticipated block-type changes and unfound write entries in TAP, etc., which are signs of missing reports from the file system. Currently, all updates are reported.

Cases Related to File-System-Consistency Properties. For cases derived from the reuse-ordering property, we created an ext3 file system with most of its i-nodes and blocks allocated to encourage reuse. Then we performed tight append/truncate and file creation/deletion loops with alternating sensitive status. We used uniquely identifiable file content to detect sensitive information leaks and found none.

For cases derived from the pointer-ordering property, we verified our ability to recover from basic failures and remove remnants of sensitive information. We also verified that the page cache prohibits unreferenced data blocks from being written to the storage.

Since the page ID component of GUIDs increases monotonically, we can use this property to detect illegal reordering of sensitive updates for the cases derived from the nonrollback property. For consolidations within a transaction, we used tight update loops with alternating sensitive modes. For consolidations across transactions, we used tight file creation/deletion loops with alternating sensitive modes. We checked all consolidation orderings for up to three requests (e.g., nonsensitive/sensitive/nonsensitive, sensitive/nonsensitive/sensitive, etc.).

6.2. TAP Verification

Information stored and subsequently returned by TAP must be accurate for correct operation of our system. Therefore, to verify TAP's operation, we enumerated a TAP state-transition table and verified its correctness via two-version programming.

State Representation: By reasoning about how TAP's information is stored, we were able to establish a state space that covers a large portion of TAP's typical operation, but at the same time is small enough to allow full enumeration and subsequent verification.

The states we chose to verify were based on two observations. First, write entry consolidation behaviors (e.g., page cache) are achieved by performing updates directly

to a given write entry. Second, the next state transition is based on inputs to current write entries of different types within a current state. In order to cover all block types, we chose states with at most one write entry of each type.

No two write entries in a given state can be of the same block type. There are nine distinct types: data, i-node, other metadata, journal copy of data, journal copy of i-node, journal copy of other metadata, copy of data, copy of i-node, and copy of other metadata. Therefore, each state contains no more than nine write entries. Additionally, each write entry has four status bits: allocated, sensitive, having reminder attached, and ready to be deleted from the journal. Thus, any state can be uniquely identified by a 9×4 matrix (2^{36} total possible states), which can be recorded with 36 bits.

State Transitions. Each TAP function call triggers a state transition based on the input arguments. For example, starting from an empty state, a `TE_report_write()` on a nonsensitive i-node will transition from the empty state (a zero matrix), say S_0, to a state S_1, where the allocated bit for the i-node block type is set to 1. If `TE_report_write()` is called again to mark the i-node as sensitive, a transition occurs from S_1 to a new state S_2, with allocated and sensitive bits set to 1s.

State-Space Enumeration: To enumerate states and transitions, we permuted all TAP interface function calls with all possible input arguments to the same set of write entries. A small range of GUIDs was used so that each write entry could have a unique GUID, but GUID collisions were allowed to test error conditions. The result is a full state-transition table populated by edges consisting of a start state, a transition, and a resulting state.

The resulting state of an edge can be classified as either a valid state, an error state, or a stop state. A valid state is determined by a valid function return code from TAP. Likewise, an error state is determined by an error function return code from TAP. All errors should cause TAP to transition back to the previous valid state. A stop state occurs when a successful return code of a function call would result in a state beyond our constraints (e.g., multiple blocks of the same type).

We created an enumeration program to systematically create a state-transition table of all reachable states (see Algorithm 1). This user-space program interacted directly with the TAP module to obtain the return code of a TAP function call and the resulting state of the given write entry.

The enumeration can be viewed as traversing a tree in breadth-first order, and the tree fanout at each level is the total number of interface call-argument combinations. Every state reachable from the empty starting state is visited only once, avoiding repeated state-space and subtree branch traversals. As a result, we explored a tree depth of 16 and located ~10K unique reachable states, or ~2.7M state transitions.

ALGORITHM 1: State-Space Enumeration of TAP Module

Input: None.
Output: State-transition table.
1. Begin with empty start state S_e and state-transition table with no transitions
2. Place S_e on FIFO
3. If FIFO not empty, retrieve current state S_c from FIFO, else return state-transition table
4. Enumerate all possible transitions from S_c to obtain a set of resulting states
For each resulting state, S_r
if S_r is valid and has never been visited, place S_r on FIFO
Update output state table with S_c to S_r transition
Goto Step 3.

In summary, we have enumerated all possible TAP states with 0 or 1 write entries and all legal sets of write entries where no two write entries of any such set have the same type.

Two-version-programming verification. We next applied n-version programming to verify the above generated state-transition table, where the probability of hitting the same bug with the same handling can be reduced as we add more versions. In this work, $n = 2$. We wrote a user-level state-transition program based on hundreds of conceptual rules (e.g., marking a write entry of any type as sensitive will set the sensitive bit to 1). The enumerated state-transition table was reconciled with the one generated by the TAP kernel module (Algorithm 1).

7. PERFORMANCE EVALUATION

We compared TrueErase's overhead on both hard-drive and NAND-flash storage. We ran PostMark to measure the overhead for metadata-intensive small-file I/Os. We also compiled OpenSSH version 5.1p1 [OpenSSH 2012] to measure the usage overhead for larger files. We ran our experiments on an Intel Pentium D CPU 2.80GHz dual-core Dell OptiPlex GX520 with 4GB DDR533. Our NAND flash was a 1GB DoC MD2203-D1024-V3-X 32-pin DIP mounted on a PCI-G DoC evaluation board, and our HDD was a Seagate Barracuda 7200.12 ST3500413AS 500GB 7200 RPM 16MB cache SATA 6.0Gb/s 3.5in internal drive. Each experiment was repeated 5 times. The 90% confidence intervals are within 23% for NAND flash and 13% for the hard drive. (The larger confidence interval for the NAND flash tests is largely due to the nature of the initial dirty state of the flash, which is detailed in Section 7.3.)

7.1. Optimization Modes

Over the course of evaluation, we found that TrueErase's performance would benefit from certain optimizations. We implemented each optimization as a running mode and explain them here.

Locality Mode. Files chosen for secure deletion can be handled either with no spatial locality or with spatial locality, which is approximated by choosing the first x% of files created under PostMark to be marked as sensitive. Tests are marked as being *random* (no spatial locality) or with *locality* (as having spatial locality).

Delay Mode. As discussed in Section 5.3.2, we implemented a delayed-deletion optimization, which defers the secure writes to data blocks until file deletion time. Tests run in this mode are marked as *delayed*. This mode is not applicable to hard drives.

Reporting Mode. For verification purposes, we originally reported all writes and deletions to the TAP module, regardless of their sensitivity. However, once the system has been verified, only sensitive writes and deletions need to be reported. We call this *selective reporting*, which is only turned on in the *extra optimization mode* (see below).

Status Caching Mode. Through code analysis, we noticed that file system i-nodes were checked frequently to determine the sensitivity of every write and delete operation, which caused other file system operations to block. We implemented our own i-node status cache to cut down on this overhead. This mode is only turned on in the *extra optimization mode* (see below).

Extra Optimization Mode. This mode is a combination of the previous two modes (selective reporting and status caching).

7.2. Benchmarks

We ran a combination of PostMark (to measure the overhead for metadata-intensive small-file I/Os) and compilation (to measure a mixed CPU and I/O workload).

Table III. PostMark Flash Operations, Times, and Overhead Percentage Compared to Base

	page reads	control-area reads	page writes	control-area writes	erases	time
Base	289K	2.27M	217K	236K	4.19K	**690 secs**
0%	1.03×	1.03×	1.03×	1.03×	1.01×	**1.03x**
1% random	4.58×	2.03×	3.29×	3.24×	3.07×	3.31×
5% random	11.1×	3.78×	7.13×	7.03×	6.99×	7.39×
10% random	14.1×	4.58×	8.83×	8.71×	8.80×	**9.24**×
1% locality	3.22×	1.64×	2.43×	2.40×	2.27×	2.43×
5% locality	7.68×	2.96×	5.29×	5.20×	4.79×	5.32×
10% locality	10.1×	3.67×	6.86×	6.74×	6.10×	**6.89**×
1% random, delayed	3.41×	1.68×	2.49×	2.47×	2.43×	2.54×
5% random, delayed	7.46×	2.89×	4.72×	4.71×	5.02×	5.09×
10% random, delayed	9.61×	3.57×	5.79×	5.81×	6.48×	**6.42**×
1% locality, delayed	2.82×	1.60×	2.17×	2.15×	2.07×	2.20×
5% locality, delayed	4.64×	2.06×	3.26×	3.25×	3.14×	3.34×
10% locality, delayed	5.85×	2.44×	4.01×	4.03×	3.87×	**4.13**×
1% locality, delayed, extra	2.43×	1.29×	1.89×	1.88×	1.83×	1.87×
5% locality, delayed, extra	4.39×	1.95×	3.15×	3.14×	2.94×	3.18×
10% locality, delayed, extra	5.47×	2.31×	3.78×	3.80×	3.64×	**3.88**×

Specifically, we modified PostMark to create and mark different percentages of files as sensitive, and a sync command (included in the elapsed time) was issued after each run to force all pending storage requests to finish. For the compilation tests, we issued the commands `make + sync` and `make clean + sync` to measure the elapsed times for compiling and cleaning OpenSSH. For the TrueErase case, we marked the `openbsd-compat` directory sensitive before issuing `make`, which caused all newly created files (e.g., .o files) in that directory to be treated sensitively. Those files account for roughly 27% of the newly generated files (8.2% of the total number of files and 4.1% of the total number of bytes after compilation).

7.3. Evaluation on NAND Flash

Both the NAND Flash PostMark and compilation tests were conducted through the ext3 file system with the INFTL module loaded. We used the default PostMark configuration with the following changes: 10K files, 10K transactions, and a read bias of 80%. Before running tests for each experimental setting, we dirtied our flash by running PostMark with 0% sensitive files enough times to trigger wear leveling. Thus, our experiments reflect a flash device operating at a realistic dirty state (although not optimal). Before running the compilation tests, we dirtied the flash in the same way as with PostMark and discarded the first run, which warms up the page cache. Tables III and IV associated with this subsection measure actual NAND flash page reads, control-area reads, page writes, control-area writes, and erases by modifying the INFTL driver to keep a tally of the actual operations.

PostMark. Table III shows that when TrueErase operates with no sensitive files, metadata tracking and queries account for 3% overhead compared to the base case. With 10% of files marked sensitive, the nonoptimized version can slow the system down by a factor of 9.2. This overhead is similar to numbers published in a prior study

Table IV. Compilation Flash Operations, Times, and Overhead Percentage Compared to Base

	page reads	control-area reads	page writes	control-area writes	erases	time
`make + sync`						
Base	25.3K	111K	22.8K	24.2K	350	89.0 secs
No optimizations	4.44×	2.82×	2.89×	2.89	2.91×	2.23×
Delayed	1.67×	1.30×	1.38×	1.40×	1.39×	1.38×
Delayed, extra	1.79×	1.39×	1.42×	1.45×	1.48×	1.43×
`make clean + sync`						
Base	1.39K	3.29K	415	482	19.2	2.6 secs
No optimizations	11.0×	10.9×	14.2×	15.6×	7.75×	9.00×
Delayed	11.4×	10.9×	13.3×	14.8×	8.57×	9.15×
Delayed, extra	11.5×	11.3×	12.4×	14.1×	8.90×	9.00×
Total						
Base	26.6K	114K	23.3K	24.7K	369	**91.6 secs**
No optimizations	4.77×	3.06×	3.10×	3.14×	3.16×	**2.42×**
Delayed	2.18×	1.58×	1.59×	1.66×	1.76×	**1.60×**
Delayed, extra	2.29×	1.68×	1.62×	1.70×	1.87×	**1.64×**

[Wei et al. 2011]. However, using all available optimizations, we are able to cut the slowdown down to a factor of 3.9.

Spatial locality has been exploited since early file system designs [McKusick et al. 1984]. Creating sensitively marked files with a high spatial locality increases the chances of similar sensitive file data and metadata to be located on the same flash block. The benefits are even more pronounced under the delayed deletion mode, where the deletion or secure write of one file will do much of the cleanup work to delete other sensitive files and metadata located on the same flash page. If a user places sensitive files inside a sensitively marked directory, we can use traditional file system spatial-locality optimizations to our advantage.

We also noticed some feedback amplification effects. Longer runs mean additional memory page flushes, which translate into more writes, which involve more reads and erases as well and thus lead to even longer times. Thus, minor optimizations, such as i-node status caching and selective reporting, can improve performance significantly.

Further, with a linear regression analysis, the total elapsed time and various numbers of flash read and write operations can generally be predicted by the number of erase operations, with an R^2 of 0.98.

OpenSSH compilations. Table IV shows that a user would experience a compilation slowdown within 1.4× under the delayed-deletion mode.

While the delayed deletion mode helped performance during a make compilation cycle, that same mode caused more erases during the corresponding clean cycle. This makes sense, since the delayed deletion mode under compilation delays more of the erasure work to occur under the clean cycle.

For the entire compilation cycle, which consists of a `make + sync` and `make clean + sync`, a user would experience an overall 1.6 to 1.7× slowdown under the delayed-deletion mode. Overall, we find the overhead is within our expectations.

7.4. Evaluation on HDD

The PostMark and compilation tests were conducted on the hard drive through the ext3 file system and with an optional device-mapper kernel module loaded. No device-mapper kernel module was loaded for the base case (unmodified kernel) runs, an empty mapper (called linear) was loaded to determine overhead, and a specialized

Table V. Postmark HDD Times (in Seconds) and Overhead Percentage Compared to Base

	No optimizations	Selective reporting	Extra
Base	**92.6 secs**	N/A	N/A
Linear	0.930×	N/A	N/A
0%	1.49×	1.28×	**1.04×**
1% random	3.52×	3.08×	2.81×
1% locality	3.25×	3.14×	2.51×
5% random	7.98×	7.30×	6.51×
5% locality	5.63×	5.10×	4.45×
10% random	10.4×	10.2×	9.71×
10% locality	7.44×	6.84×	**6.79×**

Table VI. Compilation HDD Times (in Seconds) and Overhead Percentage Compared to Base

	No optimizations	Selective reporting	Extra
`make + sync`			
Base	**71.0 secs**	N/A	N/A
Linear	1.00×	N/A	N/A
TrueErase	1.02×	1.02×	**1.01×**
`make clean + sync`			
Base	**0.512 secs**	N/A	N/A
Linear	1.00×	N/A	N/A
TrueErase	2.44×	2.54×	**2.52×**
total			
Base	**71.5 secs**	N/A	N/A
Linear	1.00×	N/A	N/A
TrueErase	1.03×	1.03×	**1.02×**

device mapper called *dm-te* (see Section 5.3.3) was loaded to determine the TrueErase overhead. We used the default PostMark configuration with the following changes: 10K files, 50K transactions, and a read bias of 80%.

PostMark. Table V shows that the addition of the empty linear device mapper caused no measurable overhead compared to the base case. This means that the presence of a device mapper itself is negligible, and any performance decreases can be attributed to the TrueErase functionality.

When TrueErase operates with no sensitive files and all possible hard-drive optimizations, metadata tracking and queries account for 4% overhead compared to the base case. Unlike the NAND flash case, TrueErase's disk performance is primarily improved through spatial locality (by 45% with 10% of files marked sensitive). The addition of the selective reporting mode and the extra optimization mode (combination of selective reporting and status caching modes) provides decreasing benefits in performance compared to nonoptimized modes as the number of securely marked files rises. A user running a metadata-intensive small-file I/O workload with 10% of the files marked as sensitive could expect slowdowns between 10× and 6.8×.

OpenSSH Compilations. Table VI shows that a user would experience a compilation slowdown of only 1% to 2%. A user would experience a slowdown within 2.52× with the delayed-deletion mode under a deletion-intensive workload. For the entire compilation cycle of `make + sync` with `make clean + sync`, a user would experience an overall slowdown within 2% to 3%.

In summary, we recommend certain optimizations for use of TrueErase with different storage types. With NAND flash, we recommend spatial locality of sensitive files, the use of an FTL which automatically tracks the overwriting of sensitive data to be deleted later (e.g., INFTL), and the TrueErase file status caching mode to ease the performance bottleneck caused by extra calls to the file system i-node-checking code. With hard drives, spatial locality of sensitive files provides a larger factor of performance gain.

Other secure deletion systems [Reardon et al. 2012] will transparently delay secure deletions into batches to increase performance. However, that design decision complicates file-system consistency, portability of the secure deletion system to other file systems, and the correctness of the system upon crash and recovery. TrueErase trades off secure deletion performance to alleviate these complications and to simplify the overall system.

8. RELATED WORK

This section discusses existing layer-specific, cross-layer, and remote secure-deletion solutions.

8.1. Layer-Specific Solutions

Various secure-deletion solutions have been proposed in the past; however, most of them operate at only one layer of the legacy storage data path (Figure 1). While layering encourages the portability of individual layers, the types of information available at each layer restrict the effectiveness of secure-deletion solutions.

8.1.1. Storage-Management Layer. At this lowest layer, one method is to encrypt all data on the storage device or partition and revoke the device's or partition's encryption key upon deletion. Many USB thumb drives provide encrypted storage, with secure deletion as a secondary goal, implemented through cryptographic mechanisms provided on the flash drive itself [Imation 2015; Kingston 2012]. Similarly, hard drives may also perform secure deletion of the entire partition or drive by employing full device encryption and revoking the key [Thibadeau 2006; Seagate 2012]. Other proposed methods may allow smaller-granularity secure deletion at the storage-management level, but do not provide a mechanism to correctly implement it. Hughes [2002] suggests implementing erasure by deleting file encryption keys at the hard-drive firmware level, but does not provide detail on how this functionality is implemented or triggered. Wei et al. [2011] propose a method of securely deleting selected flash blocks in an FTL, but they do not provide the ability to map blocks to files and metadata to support secure deletion at the file granularity. Finally, specialized disk commands may be used to overwrite data locations on a hard drive or erase all locations on a flash drive. The disk-drive Secure Erase command [Hughes 2004], defined in the ANSI ATA and SCSI interface specifications, internally overwrites and reports successful erasure of all user-accessible disk blocks.

8.1.2. File-System Layer. The file-system layer generally is unaware of details of the storage medium and, in the case of NAND flash (and even on hard drives when bad blocks are present), has no control over the physical storage location of data and metadata. Thus, a file system may issue writes of random bits to the same location, intending to perform military-grade secure deletion [U.S. NIST 2006] via *in-place* disk overwrites. Instead, the underlying storage medium can actually be a flash drive, for which in-place overwrites may not work (Section 5.3.1). File-system-layer solutions can also overlook the sensitive information left behind by common journaling mechanisms, which log information to ease recovery. Log-structured and versioning file systems may leave old and untracked versions of sensitive data on disk.

File systems might securely delete data by encrypting the data and then securely erasing the key. One solution [Peterson et al. 2005] is built on top of the versioning file system ext3cow [Peterson and Burns 2005], and is based on the all-or-nothing (AON) transform [Boyko 1999; Rivest 1997]. AON is defined as a cryptographic transform that, given only partial output, reveals nothing about its input. AON is leveraged in the secure versioning file system to make decryption impossible if one or more of the ciphertext blocks belonging to a file (or a file version) is deleted.

Two examples of data-overwriting file systems are FoSgen [Joukov et al. 2006] and Purgefs [Joukov and Zadok 2005]. Purgefs can overwrite file data and metadata when deleting or truncating a file. Alternatively, to increase efficiency, the purge-delete option can be chosen using a special file attribute. Purgefs will delete data one or more times and supports the NIST standards and all NISPOM [U.S. Department of Defense 1995] overwrite patterns. FoSgen consists of two components: a file system extension and the user-mode shred tool (Section 8.1.3). FoSgen intercepts files that require overwriting and moves them to a special directory. The shred tool, invoked either manually or periodically, eventually writes over the data in the special directory. The authors of FoSgen have also created patches to add secure-deletion functionality to the ext3 file system. In addition, Bauer and Priyantha [2001] have modified the ext2 file system for asynchronous secure deletion by passing secure-deletion information to a deletion daemon.

Other file systems are built for NAND flash and are not easily expandable to other types of storage. Lee et al. [2008] have modified YAFFS, a log-structured file system for NAND flash, to handle secure file deletion. The modified YAFFS encrypts files and stores each file's key along with the file's metadata. Whenever a file is deleted, its key is erased and the encrypted data blocks remain. Sun et al. [2008] modified YAFFS and exploited certain types of NAND flash that allow overwriting of pages to achieve secure deletion. Reardon et al. [2011] also modifies YAFFS to use a flash-chip-specific zero-overwriting technique. In addition, Reardon et al. [2012] developed the Data Node Encrypted File System (DNEFS). DNEFS modifies the flash file system UBIFS to perform secure deletion at the data node level, which is the smallest unit of reading/writing. DNEFS performs encryption and decryption of individual data nodes and relies on a key generation and deletion scheme to prevent access to overwritten or deleted data. Since UBIFS is designed for flash with scaling constraints, the approach is not as applicable for disks and larger-scale storage settings.

8.1.3. Application Layer. At the application layer, solutions are carried out by user-level programs. Given that these programs operate through file systems, they inherit the same constraints as the file-system-level solutions. Additionally, user programs that do not overwrite entire partitions have limited control over a file's metadata, and thus cannot enforce fully secure deletion, leading to the leakage of information such as file names, file sizes, etc. Three main software methods exist for overwriting sensitive data: (1) Overwrite the contents of a file, (2) delete the file normally and then overwrite all free space in the partition, and (3) erase the entire partition or disk.

The first method is quickest if only a few small files are to be securely overwritten. Many utilities, both free and commercial, are available to perform this operation. Three common UNIX utilities are shred [Shred 2012], `wipe` [Vier 2012], and `secure rm` [Gauthier and Jagdmann 2012]. These utilities will overwrite a file's content with random data for a configurable number of passes. Similarly, Apple's OS X offers a built-in file secure-erase feature [Apple, Inc. 2012] that overwrites files.

Overwriting all free space can be invoked after files have been deleted the normal way. One example is `scrub` [Garlick 2012], which erases the free space in a partition by creating a file that extends over all free blocks. A user needs to remember to remove

the file after the application is done or else use the correct "remove" flag when invoking the command. Reardon et al. [2011] introduce two user-space secure deletion methods for use on top of flash media shown to work on the YAFFS file system: purging and ballooning. Purging overwrites all free space immediately, and ballooning causes periodic secure deletion by keeping the file system full of junk data, thereby causing more frequent YAFFS garbage collection.

Erasing the entire partition or disk will securely delete all confidential information, such as data, metadata, and directory structures. One such utility is Darik's Boot and Nuke [DBAN 2012], which is a self-contained bootable medium that wipes a hard drive by filling it with random data.

8.2. Cross-Layer Solutions

A semantically smart-disk system (SDS) [Sivathanu et al. 2003] observes disk requests and deduces common file-system-level information such as block types. The File-Aware Data-Erasing Disk is an ext2-based SDS that overwrites deleted files at the file-system layer.

A type-safe disk [Sivathanu et al. 2006] directly expands the block-layer interface and the storage-management layer to perform free-space management. Using a type-safe disk, a modified file system can specify the allocation of blocks and their pointer relationships. As an example, this work implements secure deletion on ext2. Basically, when the last pointer to a block is removed, the block can be securely deleted before it is reused.

8.3. Remote and Distributed Solutions

Even on distributed and cloud-based secure deletion solutions, the action of secure deletion must still be carried out correctly on the individual systems that house the data (or the encryption keys that decrypt the data). Solutions at this level typically separate the per-file encryption keys on a different storage medium from the encrypted data. TrueErase can be used *in conjunction* with such systems to guarantee secure deletion of encryption keys.

Vanish [Geambasu et al. 2009] is a distributed secure-deletion system that aims to destroy past, archived copies of data on the Web after a user-specified amount of time without any specific action on the part of the user. It accomplishes this by encrypting information and using Shamir secret sharing to split the key into pieces so that only a configurable threshold t pieces are needed to reconstruct the decryption key. These pieces are then distributed randomly to different nodes. After a period of time, enough key pieces will vanish from the distributed nodes so that $< t$ pieces remain. However, the Vanish authors admit that, in addition to the general system, they must verify that the operating environment does not preserve information and suggest incorporating methods of sanitizing data on storage. A solution such as TrueErase could be used in conjunction with this framework. Also, Wolchok et al. [2010] found a weakness in Vanish in which a small number of computers could archive enough key pieces to reconstruct messages after they have expired.

Perlman's ephemerizer [Perlman 2005] again addresses the problem of securely erasing data after a configurable period of time. Data is encrypted, and keys are created, advertised, and destroyed through an "ephemerizer" service. The authors suggest an implementation of an ephemerizer as a remote machine in which the message keys are stored in a tamper-resistant smart card that performs cryptographic operations. It is also necessary for the ephemerizer to be authenticated with any client and communications with it protected against eavesdropping attacks.

A revocable backup system [Boneh and Lipton 1996] has a goal of securely deleting backup files without having to modify the backup media. This system encrypts backups

on a per-file basis and stores the key file on the original server. The backup file's encryption key is revoked when the backup file is to be securely deleted. The authors do not discuss how to securely erase the master key from persistent electronic storage.

Reardon et al. [2013b] present a secure deletion system at a small granularity using a B-Tree-based data structure using a small securely deleting storage medium for key storage and a large persistent storage medium. The authors store the root encryption key and record of changes on something they call a "securely-deleting medium," of which they "assume that the securely-deleting medium automatically securely deletes any data that is removed and correctly handles analog remnants and other nuances of secure deletion."

9. DISCUSSION

The following topics are discussed in this section: disk remapping and its applicability to TrueErase, the ATA8 TRIM command, cryptography and its applicability to TrueErase, and finally future expansions.

9.1. Disk Remapping

Persistent storage devices such as HDDs and NAND storage devices may remap storage locations in certain circumstances. This section discusses the remapping and its applicability to TrueErase.

Hard-Disk Drive Remapping. Over half of hard disks fail before any sector is remapped [Pinheiro et al. 2007]. As a disk drive ages, sectors may become bad. After a number of failed read or write attempts (depending on the drive's configuration), the bad sector becomes unmapped and placed on the so-called "grown list" (GLIST). The bad sector may contain sensitive information, but it is now inaccessible to the operating system. Various SMART monitoring tools [SMART 2014] can print out the number of entries in the GLIST and their corresponding addresses [Allen and Gilbert 2007].

Since bad sectors may wind up in the GLIST, TrueErase does not protect against hardware attacks that may gain access to the GLIST through unusual means. Our design assumes that many sectors (if not most) are not sensitive, and a remapping of a nonsensitive sector can safely be ignored. However, if the GLIST is determined to hold sensitive information based on the disk address, the user will need to physically destroy the device.

The SMART monitoring tools can alert a user to a remapped sector. After a hard disk drive's first reallocation, drives are over 14 times more likely to fail within 60 days than those with zero reallocation counts [Pinheiro et al. 2007]. So even if a sensitive sector were found to be remapped, the reallocation itself may indicate that the drive should soon be destroyed and replaced.

In lieu of physically destroying the device, a slightly more recoverable alternative may be to use cryptography in conjunction with TrueErase (such as proposed in Section 9.3). In this scenario, only two types of sensitive data could wind up on the GLIST: encrypted data and encryption keys. If encrypted data is put on the GLIST, the solution would be to re-encrypt the entire file. If a key is put on the GLIST, the solution would be again to re-encrypt the entire file, making sure that no old remnants of the previous encrypted file remained on storage (TrueErase secure delete + secure write).

NAND flash remapping. Similar to hard disk drives, NAND flash blocks may go bad over the lifetime of the chip. Many FTLs implement a bad block table [MICRON 2011] that is similar to the GLIST in nature and automatically remaps bad blocks.

NAND flash devices may also remap pages or blocks during garbage collection. The FTL must be aware of securely marked pages and blocks, and it must make sure that no copies of the sensitive information remain once it is copied to a new location. Our prototype FTL performs this action automatically due to the nature of its stack architecture.

9.2. Trim

The ATA8 TRIM command is implemented on some flash drives to improve performance [Shu and Obr 2007]. It allows the file system to specify blocks that are no longer in use, and the flash drive can discard them through internal garbage collection. TRIM was not meant to be a secure-deletion substitute, and it does not guarantee data deletion. Nevertheless, since the TRIM command already passes file-system deletion information to the underlying storage medium, it might seem logical to attempt to use TRIM to implement secure deletion on flash storage. However, there are a number of problems with the current implementations of TRIM:

—*Delayed deletion.* FTLs on current solid-state drives can issue the TRIM-suggested block erases at any time in the future, so it is difficult to make guarantees about if and when the specified data locations get erased. One study showed that up to 27% of blocks were recoverable on a TRIM-enabled device [King and Vidas 2011], and another study found stripes of recoverable information [Bonetti et al. 2013].
—*Metadata deletion.* The smallest unit of deletion that the TRIM command can pass down is the size of a sector. However, sometimes secure information must be erased at a smaller granularity (e.g., i-node, file name).
—*Logically addressable areas.* TRIM cannot be used as a substitute for the TrueErase `secure_write()` operation. Old copies of data from previous sensitive writes may reside in nonaddressable areas on the flash due to normal flash copy-on-write behavior. When TRIM is finally issued to the addressable areas that hold the current copy of the sensitive data, the old copies may be missed. Even TRIM in offline mode only issues TRIM requests to addressable areas.
—*File system consistency.* TRIM operations in software and firmware need to be verified to work with current file system consistency models [Patel 2009].

Reardon et al. [2013a] describe TrueErase as an enhanced version of the TRIM command that performs secure deletion. Specifically, the authors state that TrueErase "provides similar information as TRIM commands but only for all the blocks belonging to files specifically marked as sensitive." Unfortunately, this description ignores some important differences in capabilities. Unlike TRIM, TrueErase handles metadata deletion, nonlogically addressable areas on sensitive overwrites, and file system consistency issues as described above. While TrueErase is correctly categorized by Reardon et al. [2013a] as a cross-layer approach, it is not merely a secure block extension of the TRIM command.

9.3. Future Work

This section discusses multiple avenues of future work. Some future work, such as supporting multiple users, multiple file systems, and RAID, should only take additional verification effort and is discussed below. Swap files or partitions are common on modern computers, and the implications of TrueErase used in conjunction with swap are discussed. We discuss how TrueErase can enhance current cryptographic systems and how TrueErase could work on emerging storage and other types of file systems. Finally, we conclude with other avenues of future work.

Multiple Users and File Systems. TrueErase has not interfered with traditional file system semantics, with the exception of marking an i-node's directory entry as sensitive when the i-node itself is marked as sensitive. Thus, the behaviors of multiple users accessing the same sensitive file will follow traditional file system semantics (in this case, POSIX). For example, if two users have the same file open and one removes the file, it will not be deleted until its i-node access count has dropped to zero. The TrueErase prototype has also been run with multiple file systems on separate partitions, and nothing in the design precludes multiple instances of a TrueErase-enabled file system on the same persistent storage. However, verification tests would need to be built to ensure TrueErase conforms to POSIX semantics with multiple users across multiple file systems.

Multiple Disks and RAID. For TrueErase to work across multiple storage devices, the GUID address must be expanded to include a unique identifier for the storage device. In Linux, the device identifier UUID or unique device name (e.g., sda) could be appended to the GUID. In software and hardware RAID, however, only one logical device is exported to the operating system. Therefore, TrueErase identifiers could work as-is without appending a unique device identifier to the GUID.

However, caching within a RAID device/module may invalidate our assumption about if and when writes are propagated to the underlying storage device. Therefore, tests need to be conducted to ensure that write barriers are sufficient to guarantee that overwrites are flushed to the underlying storage and that any crashes during a deletion are also propagated. If these guarantees cannot be made, hardware RAID interface changes could be included, similar to the changes we propose for flash.

Swap. Upon page fault or hibernation, memory pages are written to swap. Unfortunately, memory pages may contain sensitive information. The easiest solution is to simply disable swap – average amounts of memory have risen to the point where swap is no longer mandatory for many systems. However, this would mean that systems could not enter a hibernation mode.

Since only data-path memory pages are meaningful to our framework, TrueErase lacks the scope to detect sensitive information stored in application memory. Thus, only large-granularity methods to securely delete swap information can be used. For example, TrueErase could be used to securely delete the entire swap file or partition immediately after the system resumes from hibernation. A faster alternative could involve encrypting the swap partition with a key, and TrueErase could securely delete the key file after system resume.

Emerging Storage Media. TrueErase addresses hard drives and NAND flash, which are the two storage media that dominate today's market. However, we believe that its design can also be adapted to several emerging technologies: shingled drives, phase-change memory, and MRAM [Müller et al. 2004; Sousa and Prejbeanu 2005; Wong et al. 2010]. Most of these devices do not require an FTL-like interface, so that our hard-drive techniques would be applicable. For those designs that incorporate an interface layer, we plan an approach similar to the one we use for flash.

Different Types of File Systems. Secure-deletion solutions for log-structured [Rosenblum and Ousterhout 1992] and versioning-based file systems will be more complicated. We envision that the solutions will share similar characteristics to Peterson et al. [2005]. This work has demonstrated how to apply the all-or-nothing transform [Rivest 1997] to perform secure deletion on an ext3cow versioning file system [Peterson and Burns 2005] using disks. In this case, by using TrueErase to carry out the secure-deletion operations, we can extend this solution to handle metadata erasures on both disk- and flash-based storage media.

Encryption Systems. Encryption-based secure deletion systems delete by destroying the keys of encrypted files. If encrypted data is changed or overwritten during the course of its life, it might be best to make sure old copies of the encrypted data were securely erased to prevent such attacks as two-time-pad attacks [Diesburg et al. 2008]. Thus, using encryption to perform secure deletion boils down to three challenges: (1) making sure the encryption key(s) are securely deleted upon file deletion, (2) making sure no old or overwritten copies of encryption key(s) are allowed to exist on persistent storage, and (3) making sure no old or overwritten copies of the encrypted file remain on persistent storage if the method of encryption might be vulnerable to two-time-pad attacks.

Cryptography could be added (or stacked) onto the TrueErase framework at any storage-data-path level to speed up secure deletion or enhance security. Here we provide specific example of ways TrueErase can enhance existing cryptographic systems. For the following examples, we will assume that old or overwritten copies of the encrypted file may remain on persistent storage (e.g., not vulnerable to two-time-pad attacks). If this assumption were false, the encrypted data itself would have to be marked as secure.

—*User Level.* Cryptographic solutions at this level might be homegrown. For example, a user may use gnupg [GnuPG 2014] to create encrypted files and delete a key from his or her key ring for file deletion. TrueErase could be used for this type of solution if the key file were created sensitively and no unencrypted version of the file remained on the system. If the file were decrypted before it was encrypted, the decrypted version would need to be securely created before encryption and securely deleted after encryption, using TrueErase.

—*File System.* TrueErase could add secure-deletion functionality to file systems built specifically for per-file encrypted storage [Gough 2005]. In this model, per-file keys are encrypted using a master passphrase and stored on persistent media. If the key files are created and stored securely using TrueErase, all that is theoretically necessary to securely delete the encrypted file is to securely delete the per-file encryption key file.

—*Storage Management.* Cryptographic solutions at this level typically rely on entire volumes being encrypted via key(s) encrypted by a user pass phrase. If these volume keys were created and stored securely in a file using TrueErase, all that is theoretically necessary to securely delete the encrypted volume is to securely delete the volume key file.

—*Remote / Special Secure-Deletion Medium.* Solutions at this level typically separate the per-file encryption keys on a different storage medium from the encrypted data. For example, Geambasu et al. [2009] and Perlman [2005] use dedicated server(s) for encryption keys but do not mention specifically how they guarantee keys are deleted from remote storage. Boneh and Lipton [1996] store a master encryption key somewhere offsite but again do not mention specifically how the key is destroyed on remote electronic storage. Reardon et al. [2013b] store encryption keys on a specialized securely-deleting medium. In all of these cases, keys may be stored in a sensitively marked file using TrueErase and be securely erased or overwritten as necessary.

Other future work. One avenue of future work might include implementing flash-chip-specific zero-overwriting or scrubbing routines [Sun et al. 2008; Reardon et al. 2011; Wei et al. 2011] to speed up secure deletion on NAND flash. These techniques rewrite an entire chunk of flash memory to zeros as a way to securely delete selective information without having to erase the entire erase block. However, this optimization

may make our solution less portable. Reardon et al. [2011] warn that zero-overwriting could potentially leak information through advanced forensic techniques, specifically in determining the time in which particular gates were set to zero. Wei et al. [2011] also warn that scrubbing may violate some NAND flash behaviors specified by datasheets and may alter flash pages adjacent to the page that is being overwritten.

Another avenue of future work includes tracking sensitive data between files and applications via tainting mechanisms.

10. LESSONS LEARNED AND CONCLUSION

This article presents our third version of TrueErase. Overall, we found that retrofitting security features into the legacy storage data path was more complex than we first believed.

Initially, we wanted to create a solution that would work with all popular file systems. However, we found it difficult to verify the correctness of secure deletion when working with file systems that can violate consistency properties. The verification problem became much more tractable when working with file systems with proven consistency properties.

Our earlier designs experimented with different methods of propagating information across different storage layers, such as adding new special synchronous I/O requests and sending direct flash commands from the file system. After struggling to work against the inherent asynchrony in the data path, we instead chose to work with it by associating secure-deletion information with the legacy data path flow. We also decoupled the storage-specific secure-deletion action from the secure information propagation for ease of portability to different storage types.

Additionally, we found it tricky to formulate the GUID scheme, due to multiple in-transit versions and the placement of GUIDs. For example, using only the sector number was insufficient when handling multiple in-transit updates to the same sector with conflicting sensitive status. Placing a GUID in transient data structures, such as a block I/O structure, caused complications because these structures can be split, concatenated, copied, and even destroyed before reaching storage. We solved the problem by associating the GUID with the specific memory pages that contain the data.

Our work on NAND flash would not have been possible without direct access to a flash FTL. An unfortunate trend of FTLs is that they are mostly implemented in hardware, directly on the flash device controller. To leave the door of software FTL research open, we need to encourage open environments (such as OpenSSD [INDILINX 2013]) that allow experimentation.

To summarize, we have presented the design, implementation, evaluation, and verification of TrueErase, a legacy-compatible, per-file, secure-deletion framework that can stand alone or serve as a building block for encryption- and taint-based secure deletion solutions. We have identified and overcome the challenges of specifying and propagating information across storage layers. We show that we can handle common system failures. We have verified TrueErase and its core logic via cases derived from file-system-consistency properties and state-space enumeration. We have also provided insight into the interactions of the various OS data path layers concerning deletion. Although a secure-deletion solution that can withstand diverse threats remains elusive, TrueErase is a promising step toward this goal.

ACKNOWLEDGMENTS

We thank Peter Reiher and anonymous reviewers for reviewing this article. We also thank Michael Mitchell for his benchmarking work as well as Justin Marshall and Julia Gould for their flash optimization work.

REFERENCES

B. Allen and D. Gilbert. 2007. *Bad Block HOWTO for Smartmontools*. http://smartmontools.sourceforge.net/badblockhowto.html.

Apple, Inc. 2012. Mac OS X Security Configuration for Mac OS X Version 10.6 Snow Leopard. http://images.apple.com/support/security/guides/docs/SnowLeopard_Security_Config_v10.6.pdf.

S. Bauer and N. B. Priyantha. 2001. Secure data deletion for linux file systems. In *Proceedings of the 10th USENIX Security Symposium*, 153–164.

D. Boneh and R. Lipton. 1996. A revocable backup system. In *Proceedings of the 6th USENIX Security Symposium*, 91–96.

G. Bonetti, M. Viglione, A. Frossi, F. Maggi, and S. Zanero. 2013. A comprehensive black-box methodology for testing the forensic characteristics of solid-state drives. In *Proceedings of the 29th Annual Computer Security Applications Conference*, ACM, 269–278.

V. Boyko. 1999. On the security properties of OAEP as an all-or-nothing transform. In *Proceedings of Advances in Cryptology—Crypto' 99*, 783–783.

Marcel Breeuwsma, Martien De Jongh, Coert Klaver, Ronald Van Der Knijff, and M. Roeloffs. 2009. *Forensic Data Recovery from Flash Memory*. CiteSeerX.

K. R. Butler, S. McLaughlin, and P. D. McDaniel. 2008. Rootkit-resistant disks. In *Proceedings of the 15th ACM Conference on Computer and Communications Security*, ACM, 403–416.

J. Cooke. 2007. Flash memory technology direction. *Micron Applications Engineering Document*.

G. D. Crescenzo, N. Ferguson, R. Impagliazzo, and M. Jakobsson. 1999. How to forget a secret. In *STACS 99*. C. Meinel and S. Tison (Eds.). Springer Berlin Heidelberg, 500–509.

DBAN. 2012. Darik's Boot And Nuke | Hard Drive Disk Wipe and Data Clearing. http://www.dban.org/.

S. Diesburg, C. Meyers, M. Stanovich, et al. 2012. TrueErase: Per-file secure deletion for the storage data path. In *Proceedings of the 28th Annual Computer Security Applications Conference*.

S. M. Diesburg, C. R. Meyers, D. M. Lary, and A. I. A. Wang. 2008. When cryptography meets storage. In *Proceedings of the 4th ACM International Workshop on Storage Security and Survivability*, 11–20.

W. Enck, P. McDaniel, and T. Jaeger. 2008. Pinup: Pinning user files to known applications. In *Proceedings of the Annual IEEE Computer Security Applications Conference, 2008 (ACSAC 2008)*. IEEE, 55–64.

J. Engel and R. Mertens. 2005. LogFS-finally a scalable flash file system. In *Proceedings of the 12th International Linux System Technology Conference*.

G. R. Ganger. 2001. *Blurring the line between OSes and storage devices*. Technical Report CMU-CS-01-166, Carnegie Mellon University.

S. L. Garfinkel and A. Shelat. 2003. Remembrance of data passed: A study of disk sanitization practices. *IEEE Security Privacy* 1, 1, 17–27.

J. Garlick. 2012. Scrub Utility. Retrieved from http://code.google.com/p/diskscrub/.

M. Gauthier and D. Jagdmann. 2012. Secure rm. *SourceForge*. Retrieved from http://sourceforge.net/projects/srm/.

R. Geambasu, T. Kohno, A. A. Levy, and H. M. Levy. 2009. Vanish: Increasing data privacy with self-destructing data. In *Proceedings of the 18th USENIX Security Symposium*, USENIX Association, 299–316.

GNU Privacy Guard. 2014. The GNU Privacy Guard. Retrieved from https://www.gnupg.org/.

V. Gough. 2005. Encfs. *EncFS: An Encrypted Filesystem*. Retrieved from https://vgough.github.io/encfs/.

G. Hughes. 2004. *CMRR Protocols for Disk Drive Secure Erase*. Technical report, Center for Magnetic Recording Research, University of California, San Diego.

G. F. Hughes. 2002. Wise drives [hard disk drive]. *Spectrum, IEEE* 39, 8, 37–41.

IMATION. 2015. Ironkey. Retrieved from http://www.ironkey.com.

INDILINX. 2013. The OpenSSD Project. *The OpenSSD Project*. http://www.openssd-project.org/wiki/The_OpenSSD_Project.

J. Jeong, S. S. Hahn, and S. Lee, et al. 2014. Lifetime improvement of NAND flash-based storage systems using dynamic program and erase scaling. In *Proceedings of the USENIX Conference on File And Storage Technologies (FAST)*, 61–74.

X. Jimenez, D. Novo, and P. Ienne. 2014. Wear unleveling: Improving NAND flash lifetime by balancing page endurance. In *Proceedings of the USENIX Conference on File And Storage Technologies (FAST)*, 47–59.

W. K. Josephson, L. A. Bongo, K. Li, and D. Flynn. 2010. DFS: A file system for virtualized flash storage. *Trans. Storage* 6, 3, 14, 1–14, 25.

N. Joukov, H. Papaxenopoulos, and E. Zadok. 2006. Secure deletion myths, issues, and solutions. In *Proceedings of the 2nd ACM Workshop on Storage Security and Survivability*, ACM, 61–66.

N. Joukov and E. Zadok. 2005. Adding secure deletion to your favorite file system. In *Proceedings of the 3rd IEEE International Security in Storage Workshop, 2005 (SISW'05)*. IEEE, 8, 70.

J. Katcher. 1997. *Postmark: A New File System Benchmark*. Technical Report TR3022, Network Appliance, 1997. Retrieved from www.netapp.com/tech_library/3022.html.

J. Kim, H. Shim, S. Y. Park, S. Maeng, and J. S. Kim. 2012. FlashLight: A lightweight flash file system for embedded systems. *ACM Trans. Embed. Comput. Syst. 11S*, 1, 18, 1–18, 23.

C. King and T. Vidas. 2011. Empirical analysis of solid state disk data retention when used with contemporary operating systems. *Digit. Investig.* 8, S111–S117.

KINGSTON. 2012. Secure USB Flash Drives. http://www.kingston.com/us/usb/encrypted_security.

S. R. Kleiman. 1986. Vnodes: An architecture for multiple file system types in sun UNIX. *USENIX Summer*, 238–247.

J. Lee, J. Heo, Y. Cho, J. Hong, and S. Y. Shin. 2008. Secure deletion for NAND flash file system. In *Proceedings of the 2008 ACM Symposium on Applied Computing*, ACM, 1710–1714.

S. H. Lim and K. H. Park. 2006. An efficient NAND flash file system for flash memory storage. *IEEE Trans. Comput.* 55, 7, 906–912.

R. S. Liu, C. L. Yang, and W. Wu. 2012. Optimizing NAND flash-based SSDs via retention relaxation. In *Proceedings of the USENIX Conference on File And Storage Technologies (FAST'12)*.

Y. Lu, J. Shu, and W. Zheng. 2013. Extending the lifetime of flash-based storage through reducing write amplification from file systems. In *Proceedings of the USENIX Conference on File And Storage Technologies (FAST)*, 257–270.

C. Manning. 2012. How Yaffs works | Yaffs. Retrieved from http://www.yaffs.net/documents/how-yaffs-works.

M. K. McKusick, W. N. Joy, S. J. Leffler, and R. S. Fabry. 1984. A fast file system for UNIX. *ACM Transact. Comput. Syst. (TOCS)* 2, 3, 181–197.

MICRON. 2011. *Bad Block Management in NAND Flash Memory Introduction.*

G. Müller, T. Happ, M. Kund, G. Yong Lee, N. Nagel, and R. Sezi. 2004. *Status and Outlook of Emerging Nonvolatile Memory Technologies*. 567–570.

E. B. Nightingale, K. Veeraraghavan, P. M. Chen, and J. Flinn. 2008. Rethink the sync. *ACM Trans. Comput. Syst.* 26, 3, 6, 1–6, 26.

OPENSSH. 2012. OpenSSH Homepage. *OpenSSH Homepage.* http://openssh.com/.

N. Patel. 2009. Intel pulls SSD Toolbox for killing drives under Windows 7 – Engadget. http://www.engadget.com/2009/10/27/intel-pulls-ssd-toolbox-for-killing-drives-under-windows-7/.

R. Perlman. 2005. *The Ephemerizer: Making Data Disappear.* Sun Microsystems, Inc., Mountain View, CA,.

Z. Peterson and R. Burns. 2005. Ext3cow: A time-shifting file system for regulatory compliance. *Trans. Storage* 1, 2, 190–212.

Z. N. J. Peterson, R. Burns, J. Herring, A. Stubblefield, and A. Rubin. 2005. Secure deletion for a versioning file system. In *Proceedings of the USENIX Conference on File And Storage Technologies (FAST)*, 143–154.

E. Pinheiro, W. D. Weber, and L. A. Barroso. 2007. Failure trends in a large disk drive population. In *Proceedings of the USENIX Conference on File And Storage Technologies (FAST)*, 17–23.

J. Reardon, D. Basin, and S. Capkun. 2013a. SoK: Secure data deletion. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy*.

J. Reardon, S. Capkun, and D. Basin. 2012. Data node encrypted file system: Efficient secure deletion for flash memory. In *Proceedings of the 21st USENIX Security Symposium*.

J. Reardon, C. Marforio, S. Capkun, and D. Basin. 2011. *Secure Deletion on Log-Structured File Systems.* Technical Report arXiv:1106.0917.

J. Reardon, H. Ritzdorf, D. Basin, and S. Capkun. 2013b. Secure data deletion from persistent media. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer &#38 Communications Security*, ACM, 271–284.

R. Rivest. 1997. All-or-nothing encryption and the package transform. *Fast Software Encryption*, 210–218.

M. Rosenblum and J. K. Ousterhout. 1992. The design and implementation of a log-structured file system. *ACM Trans. Comput. Syst.* 10, 1, 26–52.

P. Rusty Russell, D. Quinlan, and C. Yeoh. 1994. Filesystem hierarchy standard. *Filesystem Hierarchy Standard*. http://www.pathname.com/fhs/pub/fhs-2.3.html.

SANDISK. 2006. DiskOnChip 2000 DIP. http://jkmicro.com/DOC_2000_DIP_DS_Rev3.9.pdf.

SEAGATE. 2012. Protect your Data with Seagate Secure Self-Encrypting Drives | Seagate. http://www.seagate.com/tech-insights/protect-data-with-seagate-secure-self-encrypting-drives-master-ti/.

SHRED. 2012. Shred Linux/Unix man page. Retrieved from http://linux.about.com/library/cmd/blcmdl1_shred.htm.

F. Shu and N. Obr. 2007. *Data Set Management Commands Proposal for ATA8-ACS2.*

G. Sivathanu, S. Sundararaman, and E. Zadok. 2006. Type-safe disks. In *Proceedings of the 7th Symposium on Operating Systems Design and Implementation*, USENIX Association, 15–28.

M. Sivathanu, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, and S. Jha. 2005. A logic of file systems. In *Proceedings of the 4th USENIX Conference on File and Storage Technologies - Volume 4*, USENIX Association, 1–1.

M. Sivathanu, L. N. Bairavasundaram, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. 2004. Life or death at block-level. In *Proceedings of the 6th Conference on Symposium on Opearting Systems Design & Implementation - Volume 6*, USENIX Association, 26–26.

M. Sivathanu, V. Prabhakaran, F. I. Popovici, T. E. Denehy, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. 2003. Semantically-smart disk systems. In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies*, 73–88.

SMART. 2014. *SMART Monitoring Tools.* Retrieved from http://sourceforge.net/projects/smartmontools/.

R. C. Sousa and I. L. Prejbeanu. 2005. Non-volatile magnetic random access memories (MRAM). *Comptes Rendus Physique* 6, 9, 1013–1021.

K. Sun, J. Choi, D. Lee, and S. H. Noh. 2008. Models and design of an adaptive hybrid scheme for secure deletion of data in consumer electronics. *IEEE Trans. Consumer Electronics*, 54, 1, 100–104.

R. Thibadeau. 2006. Trusted computing for disk drives and other peripherals. *Security Privacy, IEEE* 4, 5, 26–33.

S. C. Tweedie. 1998. Journaling the linux ext2fs filesystem. In *Proceedings of the 4th Annual Linux Expo.*

UBIFS. 2008. Unsorted Block Image File System. http://www.linux-mtd.infradead.org/doc/ubifs.html.

United States Census Bureau. 2014. Computer and Internet Use in the United States: 2013. *Computer and Internet Use in the United States: 2013.* Retrieved from http://www.census.gov/content/dam/Census/library/publications/2014/acs/acs-28.pdf.

U.S. Department of Defense. 1995. National Industrial Security Program Operating Manual 5220.22-M. http://www.usaid.gov/policy/ads/500/d522022m.pdf.

U.S. Department of Health & Human Services. 1996. *Health Insurance Portability and Accountability Act of 1996.* http://www.hhs.gov/ocr/privacy/hipaa/administrative/statute/hipaastatutepdf.pdf.

U.S. Department of Health & Human Services. 2009. HITECH Act Enforcement Interim Final Rule. http://www.hhs.gov/ocr/privacy/hipaa/administrative/enforcementrule/hitechenforcementifr.html.

U.S. Government. 1999. Gramm-Leach-Bliley Act. http://www.gpo.gov/fdsys/pkg/PLAW-106publ102/html/PLAW-106publ102.htm.

U.S. Government. 2002. Sarbanes-Oxley Act of 2002. http://www.gpo.gov/fdsys/pkg/PLAW-107publ204/html/PLAW-107publ204.htm.

U.S. Government. 2003. Fair and Accurate Credit Transactions Act of 2003. http://www.gpo.gov/fdsys/pkg/PLAW-108publ159/html/PLAW-108publ159.htm.

U.S. NIST. 2006. Special Publication 800-88: Guidelines for Media Sanitization. http://csrc.nist.gov/publications/nistpubs/800-88/NISTSP800-88_with-errata.pdf.

A. Venkatesh, D. Dunkle, and A. Wortman. 2011. *Evolving Patterns of Household Computer Use: 1999–2010.* University of California, Irvine, Center for Research on Information Technology and Organizations.

T. Vier. 2012. Wipe: Secure File Deletion. http://wipe.sourceforge.net/.

M. Wei, L. M. Grupp, F. E. Spada, and S. Swanson. 2011. Reliably erasing data from flash-based solid state drives. In *Proceedings of the 9th USENIX Conference on File And Stroage Technologies (FAST)*. USENIX Association, 8–8.

S. Wolchok, O. S. Hofmann, and N. Heninger, et al. 2010. Defeating vanish with low-cost sybil attacks against large DHTs. *NDSS*.

H. S. P. Wong, S. Raoux, and S. Kim, et al. 2010. Phase change memory. *Proc. IEEE* 98, 12, 2201–2227.

D. Woodhouse. 2001. JFFS: The journalling flash file system. *Ottawa Linux Symposium.*

D. Woodhouse. 2008. Jffs2: The journalling flash file system, version 2. http://sourceware.org/jffs2/.

C. Wright, D. Kleiman, and R. S. Sundhar, S. 2008. Overwriting hard drive data: The great wiping controversy. In *Proceedings of the 4th International Conference on Information Systems Security*, Springer-Verlag, 243–257.

G. Wu and X. He. 2012. Reducing SSD read latency via NAND flash program and erase suspension. In *Proceedings of the 10th USENIX Conference on File And Stroage Technologies (FAST)*. 10.

M. Wu and W. Zwaenepoel. 1994. eNVy: A non-volatile, main memory storage system. In *Proceedings of the 6th International Conference on Architectural Support for Programming Languages and Operating Systems*, ACM, 86–97.

Y. Zhang, L. P. Arulraj, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. 2012. De-indirection for flash-based SSDs with nameless writes. In *Proceedings of the 10th USENIX Conference on File And Stroage Technologies (FAST)*, 1.

K. Zhao, W. Zhao, H. Sun, T. Zhang, X. Zhang, and N. Zheng. 2013. LDPC-in-SSD: Making advanced error correction codes work effectively in solid state drives. In *Proceedings of the 11th USENIX Conference on File And Stroage Technologies (FAST)*, 243–256.

Received July 2014; revised July 2015; accepted October 2015